

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

GURAJADA VIZIANAGARAM

VIZIANAGARAM – 533003

ANDHRA PRADESH

VOLUNTEER MANAGEMENT APP

By

VILLURI JAHNAVI

(Regd. No. 24VV1A1260)

Under the supervision of

MRS. MADHUMITHA CHANDA

MRS. M.GEETHA MADHURI YADAV

Department of Information Technology

Vizianagaram – 533003



JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Regd.No : 24VV1A1260

CERTIFICATE

This is to certify that this is a bonafide record of practical work done by **VILLURI JAHNAVI** of IIIrd B.Tech 1st Semester Class in User Interface Design using Flutter Lab during the year 2026-27.

No.of Tasks Completed and Certified: 11

Lecture In-Charge **Head of The Department**

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Website:www.jntugvcev.edu.in

Subject Name: USER INTERFACE DESIGN USING FLUTTER Subject Code: R233112ES01

Academic Year: 2026 **Regulation: R23**

COURSE OUTCOMES

NBA Subject Code	Course Outcomes	
R233112ES01	CO1	Learns to Implement Flutter Widgets and Layouts.
	CO2	Understands Responsive UI Design and with Navigation in Flutter.
	CO3	Knowledge on Widgets and customize widgets for specific UI elements, Themes
	CO4	Understand to include animation apart from fetching data.

CO-PO Mapping

Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Outcomes		Program Outcomes (POs)														
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
R233112ES01	CO1	3	2	3	2	3	1	1	1	2	2	1	2	3	3	2
	CO2	3	3	3	2	3	1	2	2	2	3	2	2	3	3	2
	CO3	3	2	3	2	3	2	1	1	2	2	1	2	3	2	2
	CO4	3	3	3	3	3	2	1	2	3	3	2	3	3	3	3

Enter correlation levels 1,2 and 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-"

Signature of the Course Instructor

CONTENTS

S.No.	Date	Experiment Name	Page No.	Marks	Signature
1(a)		Installation of Flutter and Dart SDK	1		
1(b)		Dart Programming Basics	2		
2(a)		Explore Flutter Widgets	3		
2(b)		Layouts Using Row, Column, and Stack	4		
3(a)		Responsive User Interface	5		
3(b)		Media Queries and Breakpoints	6		
4(a)		Navigation Using Navigator	7		
4(b)		Navigation Using Named Routes	8		
5(a)		Stateful and Stateless Widgets	9		
5(b)		State Management Using setState	10		
6(a)		Creating Custom Widgets	11		
6(b)		Themes and Custom Styles	12		
7(a)		Creating Forms with Input Fields	13		
7(b)		Form Validation and Error Handling	14		
8(a)		Implementing Animations	15		
8(b)		Different Animation Types	16		
9(a)		Connecting to a REST API	17		
9(b)		Displaying Fetched API Data	18		
10(a)		Unit Testing in Flutter	19		
10(b)		Debugging a Flutter Application	20		
11		Case Study: Volunteer Connect (Event Volunteer Management System)	21		

Experiment 1(a): Install Flutter and Dart SDK

Aim

To install Flutter and the Dart SDK and verify the set-up using commands relevant to the Volunteer Connect project.

Description

Flutter is the cross-platform UI toolkit used to build the Volunteer Connect front end, and the Dart SDK supplies the language and tooling. After installation, the Flutter command-line tools confirm the installed versions, list the available devices, download the packages declared in pubspec.yaml (http, google_fonts, shared_preferences, sqflite and sqflite_common_ffi) and launch the application. Volunteer Connect was run on the Windows desktop target from VS Code; Chrome and Edge are also detected as web targets. The Node.js backend must be started separately (node server.js in the backend folder) before login and event data work.

Code

```
flutter --version
dart --version
flutter doctor
flutter pub get
flutter devices
flutter run -d windows
```

Output

The commands below were run on the development computer. Flutter 3.44.8 with Dart 3.12.2 is installed and the Windows desktop, Chrome and Edge devices are available.

```
Flutter 3.44.8 • channel stable • https://github.com/flutter/flutter.git
Framework • revision 058e0af2c2 (3 months ago) • 2026-07-23 10:56:21 -0700
Engine • hash 13ffd72b2f9a5ca4db2a74ea52d5353ec2e8f939 (revision 0cd610717b) (3 months ago) • 2026-07-23
16:11:34.000Z
Tools • Dart 3.12.2 • DevTools 2.57.0
Dart SDK version: 3.12.2 (stable) (Tue Jun 9 01:11:39 2026 -0700) on "windows_x64"


Found 3 connected devices:
  Windows (desktop) • windows • windows-x64    • Microsoft Windows [Version 10.0.26200.9457]
  Chrome (web)      • chrome  • web-javascript • Google Chrome 155.0.8059.85
  Edge (web)        • edge    • web-javascript • Microsoft Edge 156.0.4314.42
```

Experiment 1(b): Write a simple Dart program to understand language basics

Aim

To write a simple Dart program to understand language basics using examples relevant to the Volunteer Connect project.

Description

Dart is the language used to describe widgets, application logic and data models in Flutter. Constants store values such as the application name and the two account roles, lists hold the enrollment statuses used by the app (pending, approved, rejected), and a map holds sample event data. A function with string interpolation builds the label shown on an event card ("Full" or "N spots"). For and conditional expressions process the values just as the app does while displaying events. The program was executed with dart run and the printed output is shown below.

Code

```
void main() {
  const appName = 'Volunteer Connect';
  const roles = ['Volunteer', 'Head Volunteer'];
  const statuses = ['pending', 'approved', 'rejected'];
  final event = {'name': 'Tech Fest 2026', 'spots': 50, 'enrolled': 12};
  String spotsLabel(int n) => n == 0 ? 'Full' : '$n spots';
  print('Project: $appName');
  for (final role in roles) { print('Role: $role'); }
  for (final s in statuses) { print('Enrollment status: ${s.toUpperCase()}'); }
  final left = (event['spots'] as int) - (event['enrolled'] as int);
  print('${event['name']} -> ${spotsLabel(left)}');
}
```

Output

Dart variables, collections, functions, loops and string interpolation form the basis of the models and display logic in Volunteer Connect. Output of the program:

```
Project: Volunteer Connect
Role: Volunteer
Role: Head Volunteer
Enrollment status: PENDING
Enrollment status: APPROVED
Enrollment status: REJECTED
Tech Fest 2026 -> 38 spots
```

Experiment 2(a): Explore Flutter widgets (Text, Image, Container, etc.)

Aim

To explore Flutter widgets (Text, Image, Container, etc.) using examples relevant to the Volunteer Connect project.

Description

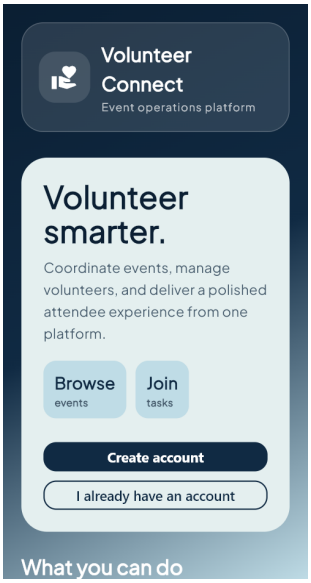
Flutter interfaces are built by composing widgets rather than by writing separate native layouts for each platform. In Volunteer Connect, Text displays headings and event details; Container controls padding, background and rounded decoration; Icon and CircleAvatar present quick-action symbols; Image.network loads the event artwork; and FilledButton and OutlinedButton trigger navigation. Cards and icons make the Welcome and Home screens easy to scan, and consistent colours come from the AppColors class. The snippet is the header card of the Welcome screen.

Code

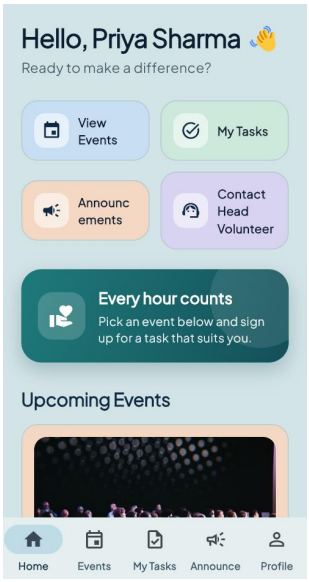
```
Container(
  padding: const EdgeInsets.all(18),
  decoration: BoxDecoration(
    color: Colors.white.withValues(alpha: 0.12),
    borderRadius: BorderRadius.circular(24),
  ),
  child: Row(children: [
    const Icon(Icons.volunteer_activism, color: Colors.white, size: 30),
    const SizedBox(width: 12),
    const Text('Volunteer Connect',
      style: TextStyle(fontSize: 21, fontWeight: FontWeight.w800, color: Colors.white)),
  ]),
)
```

Output

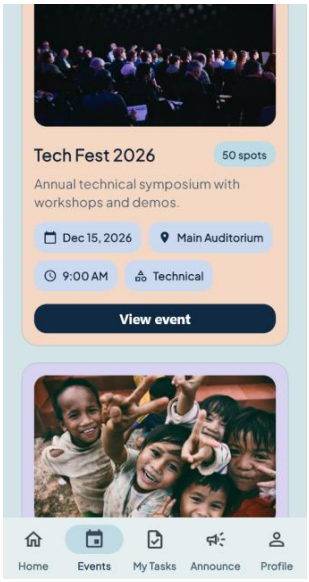
The Welcome and Home screens are composed from widgets. Text shows information, Container decorates sections, Icon and Image add visual cues and buttons trigger actions.



Welcome screen



Volunteer Home screen



Event card with image

Experiment 2(b): Implement layouts using Row, Column and Stack

Aim

To implement layouts using Row, Column and Stack using examples relevant to the Volunteer Connect project.

Description

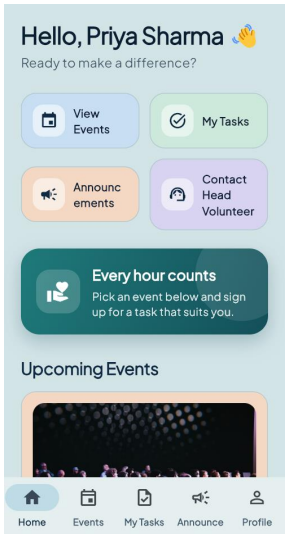
Row places widgets horizontally and Column places them vertically; both are used throughout the dashboards and forms. Stack positions widgets on top of one another. In Volunteer Connect, each quick-action card on the Home screen is a Row of an icon tile and an Expanded label, a Column lists the screen heading and cards, and the gradient SplashCard ("Every hour counts", "Lead the way") is a Stack that places a translucent circle and an optional image behind its text row.

Code

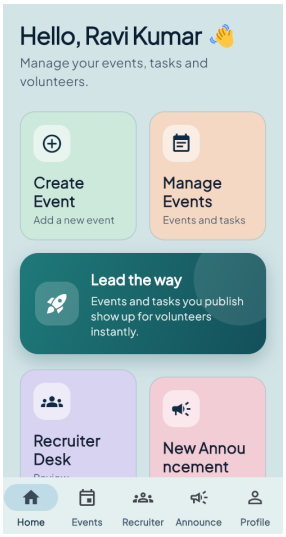
```
// Row: icon tile + label (quick action card)
Row(children: [
  Container(width: 38, height: 38,
    decoration: BoxDecoration(color: Colors.white.withValues(alpha: 0.55),
      borderRadius: BorderRadius.circular(12)),
    child: Icon(icon, color: AppColors.primary, size: 22)),
  const SizedBox(width: 10),
  Expanded(child: Text(title, style: const TextStyle(fontSize: 13))),
])
// Stack: decorative circle behind the content
Stack(children: [
  Positioned(right: -24, top: -24, child: Container(width: 110, height: 110,
    decoration: BoxDecoration(shape: BoxShape.circle,
      color: Colors.white.withValues(alpha: 0.1)))),
  Padding(padding: const EdgeInsets.all(18), child: Row(children: [ /* icon + text */ ])),
])
```

Output

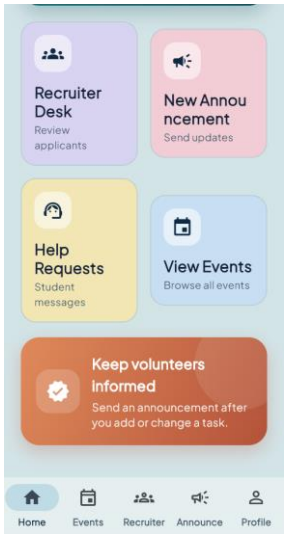
Row, Column and Stack combine to produce the Home layout: tiles are arranged in rows, sections in a column, and the banner cards use a Stack for the overlapping decoration.



Row/Column: quick actions



Stack: banner card (Head Home)



Row/Column: Head dashboard tiles

Experiment 3(a): Design a responsive UI for different screen sizes

Aim

To design a responsive UI for different screen sizes using examples relevant to the Volunteer Connect project.

Description

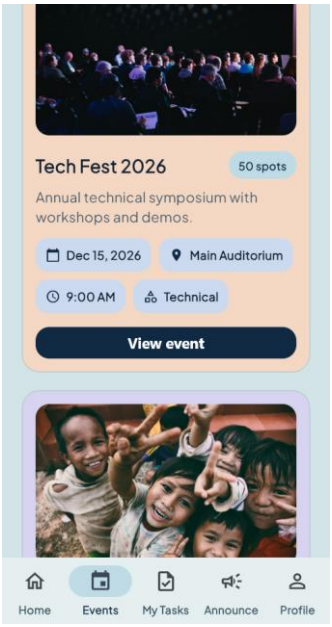
Responsive design makes an interface adapt to the space available. Volunteer Connect screens are built from flexible widgets rather than fixed sizes: SingleChildScrollView lets any page scroll when its content is taller than the window, Expanded and double.infinity widths let buttons and fields fill the available width, and Wrap moves the date, location, time and category chips onto new lines when they do not fit. The same screens therefore remain usable in a narrow, phone-like window and in a wide desktop window.

Code

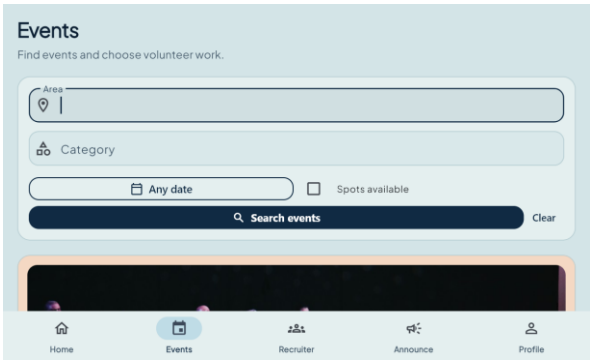
```
SingleChildScrollView(
  physics: const AlwaysScrollableScrollPhysics(),
  padding: const EdgeInsets.all(20),
  child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
    Wrap(spacing: 8, runSpacing: 8, children: [
      _EventInfoChip(icon: Icons.calendar_today, label: date),
      _EventInfoChip(icon: Icons.location_on, label: location),
    ]),
    SizedBox(width: double.infinity,
      child: FilledButton(onPressed: openDetails, child: const Text('View event'))),
  ]),
)
```

Output

The event card below is shown in a narrow window (chips wrap onto two lines) and the Events page in a wide window (fields and cards stretch to fill the width).



Narrow window



Wide window

Experiment 3(b): Implement media queries and breakpoints for responsiveness

Aim

To implement media queries and breakpoints for responsiveness using examples relevant to the Volunteer Connect project.

Description

Flutter does not use CSS media queries; MediaQuery and LayoutBuilder provide the window size and layout constraints, from which breakpoints can be defined to change the number of columns. The current Volunteer Connect build uses a single-column layout at every width, so a wider window simply stretches the cards (see the output). The code below is the proposed extension for the Events page: it chooses one, two or three columns from the window width, and is given as the next step for the project rather than as a feature that is already implemented.

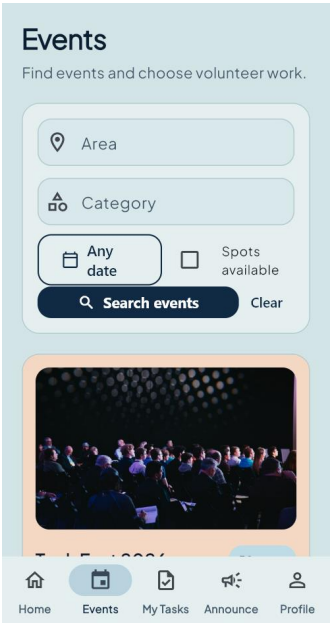
Code (proposed extension)

```
int columnsFor(double width) {
  if (width >= 900) return 3;
  if (width >= 600) return 2;
  return 1;
}

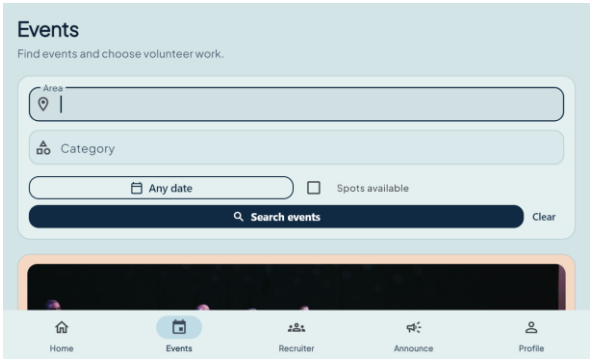
@override
Widget build(BuildContext context) {
  return LayoutBuilder(builder: (context, constraints) {
    final columns = columnsFor(constraints.maxWidth);
    return GridView.count(crossAxisCount: columns, childAspectRatio: 0.85,
      children: events.map((e) => EventCard.fromJson(e)).toList());
  });
}
```

Output

Measured behaviour of the current build: the layout adapts by stretching, not by re-arranging columns. The two captures show the same Events page at two window widths.



Narrow window



Wide window

Experiment 4(a): Set up navigation between screens using Navigator

Aim

To set up navigation between screens using Navigator using examples relevant to the Volunteer Connect project.

Description

Navigator manages the route stack used to move between Flutter screens. Navigator.push opens a destination and the AppBar back arrow pops it. In Volunteer Connect, the Welcome screen pushes the Signup and Login screens, and after a successful login Navigator.pushReplacement swaps the Login screen for the home screen so that the back button does not return to the login form. The event cards push the Event Details screen with the selected event data.

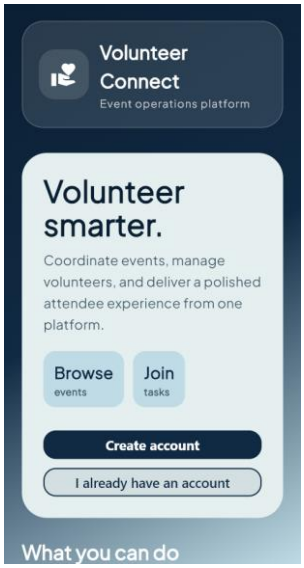
Code

```
// Welcome screen -> Login
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const LoginScreen()),
);

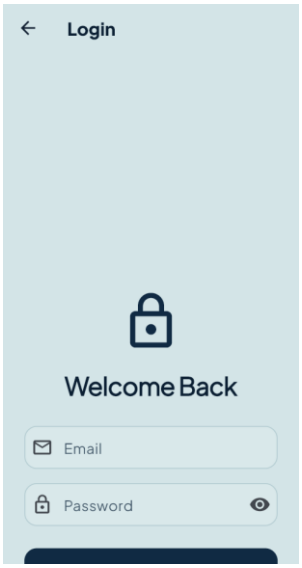
// After a successful login
Session.setUser(user);
Navigator.pushReplacement(
  context,
  MaterialPageRoute(builder: (context) => VolunteerHomeScreen(user: user)),
);
```

Output

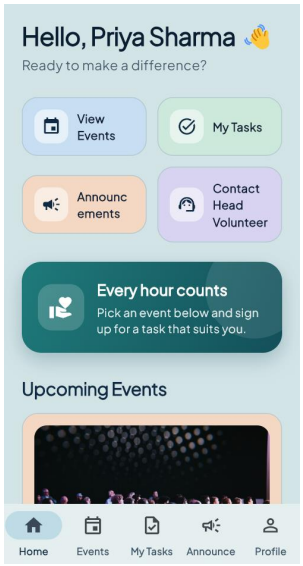
Pressing "I already have an account" opens the Login screen, and pressing LOGIN with valid credentials replaces it with the Home screen.



Welcome



Login (pushed)



Home (pushReplacement)

Experiment 4(b): Implement navigation with named routes

Aim

To implement navigation with named routes using examples relevant to the Volunteer Connect project.

Description

Named routes assign a string identifier to each destination in MaterialApp so that navigation code refers to the identifier instead of building the page each time. The current Volunteer Connect build navigates with MaterialPageRoute directly (Experiment 4(a)) and uses an IndexedStack with a NavigationBar to switch between the five main tabs. The code below shows how the same screens could be registered as named routes; it is a proposed variant and is not part of the submitted source.

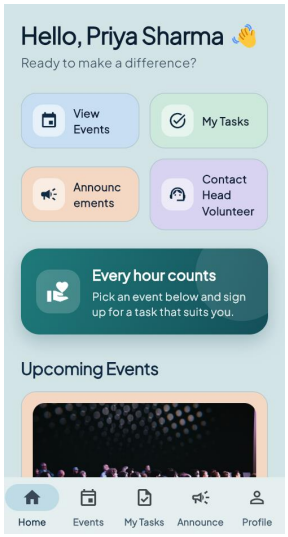
Code (proposed variant)

```
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (context) => const SplashScreen(),
    '/welcome': (context) => const WelcomeScreen(),
    '/login': (context) => const LoginScreen(),
    '/signup': (context) => const SignupScreen(),
  },
)

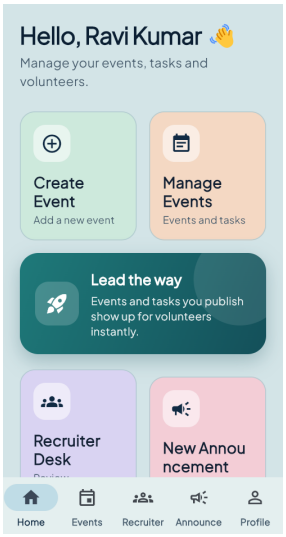
Navigator.pushNamed(context, '/login');
Navigator.pushReplacementNamed(context, '/welcome');
```

Output

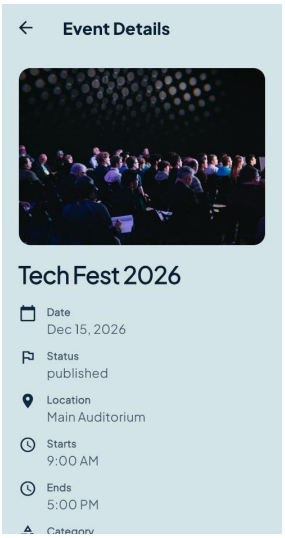
The bottom NavigationBar of the real application (Home, Events, My Tasks, Announce, Profile) is the in-app tab navigation, with the Head Volunteer variant replacing My Tasks by Recruiter.



Volunteer tabs



Head Volunteer tabs



Pushed Event Details

Experiment 5(a): Learn about stateful and stateless widgets

Aim

To learn about stateful and stateless widgets using examples relevant to the Volunteer Connect project.

Description

StatelessWidget is suitable for UI that is built only from the values supplied to it, while StatefulWidget is used when a screen must change in response to time, input or data. In Volunteer Connect, FeatureCard, StepCard and EventCard are stateless because they only display their constructor arguments. SplashScreen, LoginScreen, SignupScreen and the tab pages are stateful: the splash screen owns a Timer, the login form owns text controllers and a loading flag, and the Events page owns the filter values and the list loaded from the server.

Code

```
class FeatureCard extends StatelessWidget {
  final IconData icon; final String title; final String description;
  const FeatureCard({super.key, required this.icon, required this.title, required
this.description});
}

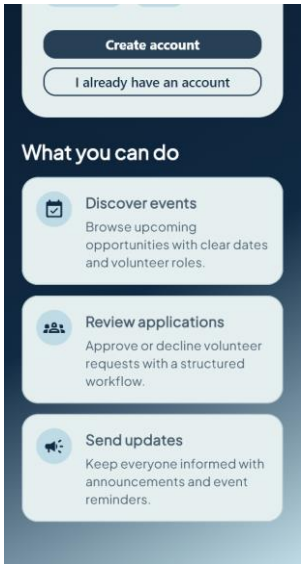
class _SplashScreenState extends State<SplashScreen> {
  @override
  void initState() {
    super.initState();
    Timer(const Duration(seconds: 2), () {
      if (!mounted) return;
      Navigator.pushReplacement(context, MaterialPageRoute(builder: (_) => const
WelcomeScreen()));
    });
  }
}
```

Output

The splash screen (stateful) waits two seconds and then moves to the Welcome screen, whose feature cards (stateless) display fixed content.



SplashScreen (stateful)



FeatureCard list (stateless)

Experiment 5(b): Implement state management using setState and Provider

Aim

To implement state management using setState and Provider using examples relevant to the Volunteer Connect project.

Description

State management keeps the interface synchronised with values that change during use. setState suits local state inside one StatefulWidget: Volunteer Connect uses it for the selected tab, the loading flag of the login button, the password visibility toggle and the selected account role. State shared by several screens, namely the logged-in user, is held in a small static Session class that stores the user returned by the login API and answers role questions such as isHeadVolunteer. The Provider package is not used in this project (it is not listed in pubspec.yaml); Session plays that role at a smaller scale.

Code

```
// Local state with setState
void _selectTab(int index) {
  setState(() => selectedIndex = index);
}

setState(() { isLoading = true; });
final user = await ApiService.loginUser(email: email, password: password);

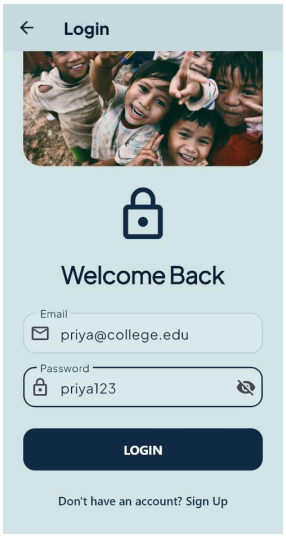
// Shared state: Session (lib/services/session.dart)
class Session {
  static Map<String, dynamic>? currentUser;
  static void setUser(Map<String, dynamic>? user) { currentUser = user; }
  static String get role => (currentUser?['role'] ?? '') as String;
}
```

Output

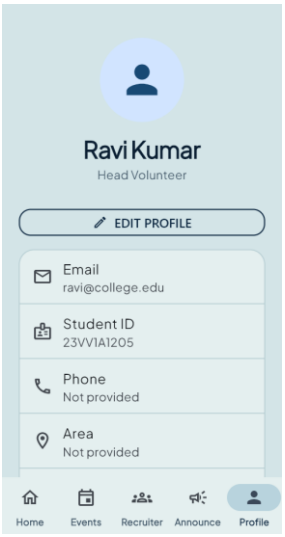
Selecting a role card, toggling the password eye icon and switching tabs all rebuild only the affected screen through setState, and the stored Session decides which tab set (Volunteer or Head Volunteer) is shown after login.



Selected role (setState)



Login form state



Role read from Session

Experiment 6(a): Create custom widgets for specific UI elements

Aim

To create custom widgets for specific UI elements using examples relevant to the Volunteer Connect project.

Description

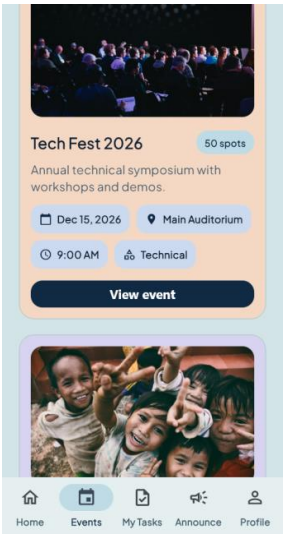
Custom widgets wrap repeated UI patterns into reusable components. Volunteer Connect defines EventCard (event image, name, spots badge, chips and the View event button), SplashCard (gradient message banner), GradientBadge (coloured status label), EventArtwork (network image with fallback), LoadErrorState (message with a Retry button) and FadeSlideIn (entry animation). Each is configured through constructor arguments, so the same widget is reused on the Home page, the Events page and the Head Volunteer screens.

Code

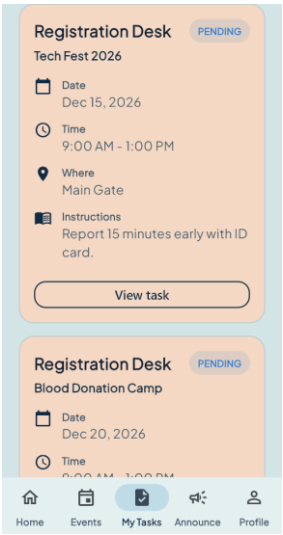
```
class GradientBadge extends StatelessWidget {
  final String label; final Color color;
  const GradientBadge({super.key, required this.label, required this.color});
  @override
  Widget build(BuildContext context) {
    return Container(padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      decoration: BoxDecoration(
        gradient: LinearGradient(colors: [color.withValues(alpha: 0.18), color.withValues(alpha: 0.08)]),
        borderRadius: BorderRadius.circular(24)),
      child: Text(label, style: TextStyle(color: color, fontWeight: FontWeight.w700)));
  }
}
```

Output

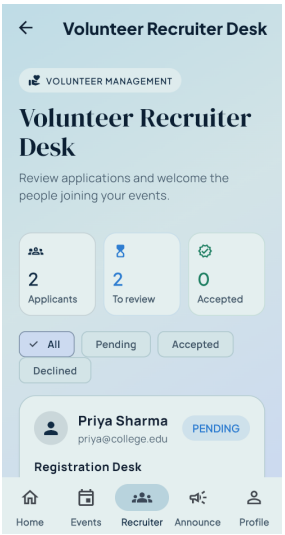
The custom EventCard and the status badges appear on the Events, My Tasks and Recruiter screens with consistent styling.



EventCard



Task card + status badge



Recruiter cards

Experiment 6(b): Apply styling using themes and custom styles

Aim

To apply styling using themes and custom styles using examples relevant to the Volunteer Connect project.

Description

ThemeData defines common colours, typography and component styling for the whole application. Volunteer Connect keeps all style values in lib/theme/app_theme.dart: AppColors (navy primary, soft teal background, pastel card tints), AppSpacing, AppRadius, AppTextStyles and an AppTheme.light theme built with ColorScheme.fromSeed and the Plus Jakarta Sans font from google_fonts. The app uses a single light theme; the Settings screen states "Light Theme Only", and a unit test confirms the brightness and key colours.

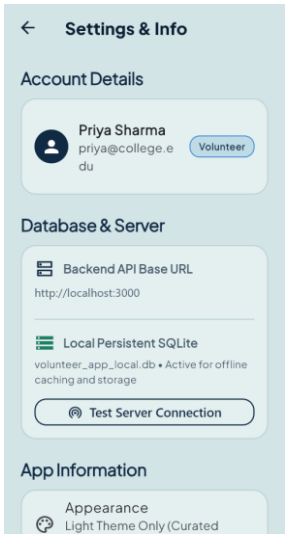
Code

```
class AppColors {
  static const primary = Color(0xFF102A43);
  static const background = Color(0xFFD3E4E7);
  static const surface = Color(0xFFE4EFEF);
  static const success = Color(0xFF1E7F62);
  static const danger = Color(0xFFD14B5B);
}

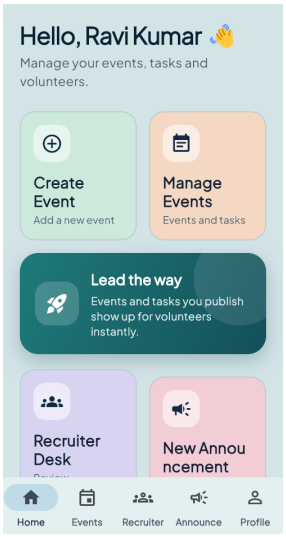
ThemeData(
  useMaterial3: true,
  brightness: Brightness.light,
  scaffoldBackgroundColor: AppColors.background,
  colorScheme: ColorScheme.fromSeed(seedColor: AppColors.primary, primary: AppColors.primary,
    surface: AppColors.surface, secondary: AppColors.accent, error: AppColors.danger),
)
```

Output

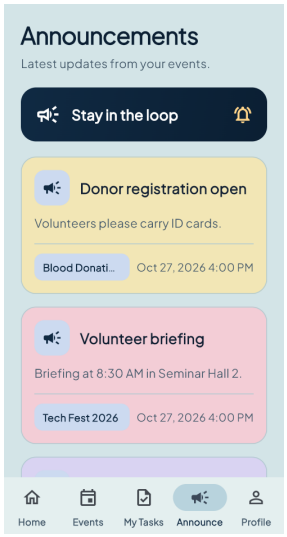
Every screen shares the same palette, rounded cards and typography because they read from the single theme file.



Settings & Info



Themed Head Home



Themed announcement cards

Experiment 7(a): Design a form with various input fields

Aim

To design a form with various input fields using examples relevant to the Volunteer Connect project.

Description

A form collects structured information from the user. The Head Volunteer "Add Event" screen contains text fields for the event name, description, category, location, area and address, read-only date and time pickers, numeric fields such as the number of volunteers, and optional fields for roles, meeting point, reporting instructions, contact phone and a poster image URL. Each input is a TextFormField with a TextEditingController, a prefix icon and an outlined border, created through one reusable _field helper and wrapped in a Form that uses a GlobalKey.

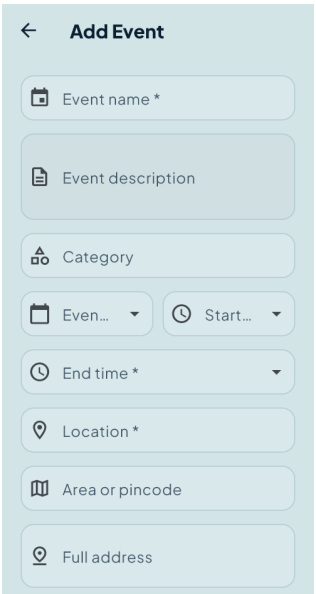
Code

```
final _formKey = GlobalKey<FormState>();

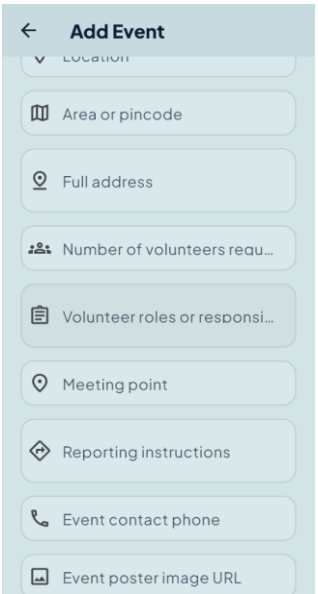
Form(
  key: _formKey,
  child: ListView(padding: const EdgeInsets.all(20), children: [
    _field(_name, 'Event name *', icon: Icons.event, validator: ...),
    _field(_location, 'Location *', icon: Icons.location_on_outlined, validator: ...),
    _field(_area, 'Area or pincode', icon: Icons.map_outlined),
    // date / start time / end time pickers, volunteers needed, roles, ...
  ]),
)
```

Output

The Add Event form shows required fields (marked with *) followed by optional details; the second capture shows the lower part of the same scrolling form.



Add Event: top



Add Event: lower fields

Experiment 7(b): Implement form validation and error handling

Aim

To implement form validation and error handling using examples relevant to the Volunteer Connect project.

Description

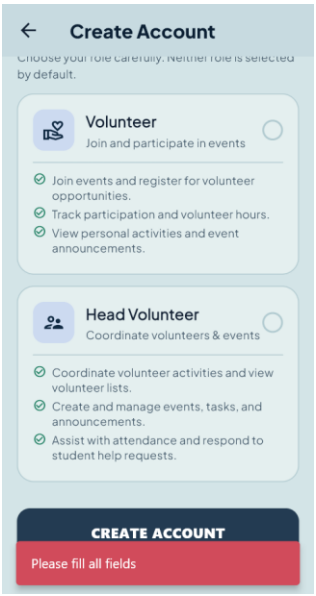
Validation checks that required information is present and acceptable before it is sent to the server. The Signup screen checks that all fields are filled, that the email contains "@" and ".", that the password has at least six characters and that a role has been chosen; failures are reported with a red SnackBar. The Login screen checks for empty fields and shows server errors, including a timeout message when the Node.js backend is not running. The Add Event form uses FormField validators and calls _formKey.currentState!.validate() before saving, and the backend repeats the checks and returns HTTP 400 for invalid data.

Code

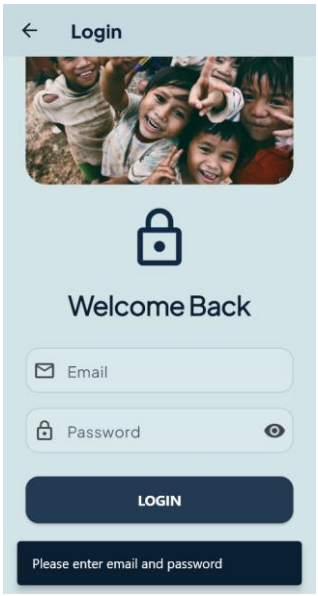
```
 1  if (name.isEmpty || email.isEmpty || studentId.isEmpty || password.isEmpty) {
 2    ScaffoldMessenger.of(context).showSnackBar(const SnackBar(content: Text('Please fill all
fields'), backgroundColor: AppColors.danger));
 3    return;
 4  }
 5  if (!email.contains('@') || !email.contains('.')) { /* 'Please enter a valid email address' */ }
 6  if (password.length < 6) { /* 'Password must contain at least 6 characters' */ }
 7
 8  validator: (value) => value == null || value.trim().isEmpty ? 'Enter an event name' : null,
 9  if (!_formKey.currentState!.validate()) return;
```

Output

Submitting the empty Signup form shows "Please fill all fields", and pressing LOGIN with empty fields shows "Please enter email and password".



Signup validation



Login validation

Experiment 8(a): Add animations to UI elements using Flutter animation framework

Aim

To add animations to UI elements using Flutter animation framework using examples relevant to the Volunteer Connect project.

Description

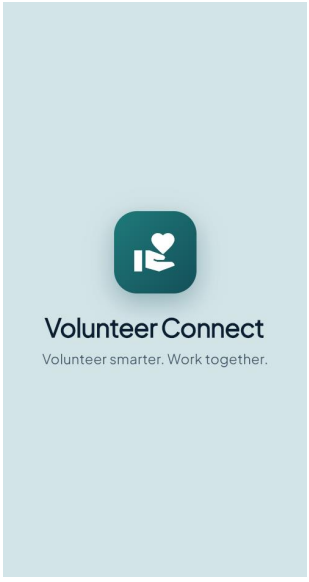
Animation gives feedback and makes screens feel smoother when used with restraint. Volunteer Connect uses implicit animation through TweenAnimationBuilder. The splash logo scales in with an elastic curve and fades in, and the reusable FadeSlideIn widget makes headings, quick-action cards and event cards fade and slide up with a small staggered delay based on their index. The theme also sets FadeForwardsPageTransitionsBuilder for all platforms, so route changes fade forward.

Code

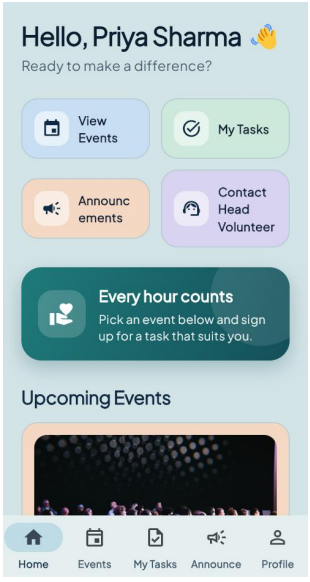
```
class FadeSlideIn extends StatelessWidget {
  // child, index (stagger), duration, offset
  @override
  Widget build(BuildContext context) {
    final delay = Duration(milliseconds: 70 * index.clamp(0, 8));
    final total = duration + delay;
    return TweenAnimationBuilder<double>(
      tween: Tween(begin: 0, end: 1), duration: total,
      curve: Interval(delay.inMilliseconds / total.inMilliseconds, 1, curve: Curves.easeOutCubic),
      builder: (context, value, child) => Opacity(opacity: value,
        child: Transform.translate(offset: Offset(0, (1 - value) * offset), child: child)),
      child: child);
  }
}
```

Output

The splash logo animates into place when the application starts, and the Home page contents fade and slide in one after another when the screen opens.



Splash: scale + fade



Home: staggered FadeSlideIn

Experiment 8(b): Experiment with fade, slide and other animations

Aim

To experiment with fade, slide and other animations using examples relevant to the Volunteer Connect project.

Description

Fade transitions change opacity and slide effects move a widget into view; other implicit widgets animate size, colour and scale. Volunteer Connect combines several of these: the role cards on the Signup screen use AnimatedContainer to animate the border, fill colour and shadow when a role is selected, and PressableScale uses AnimatedScale so that a tappable card shrinks to 97% while it is pressed. The same FadeSlideIn widget from Experiment 8(a) provides the combined fade and slide effect.

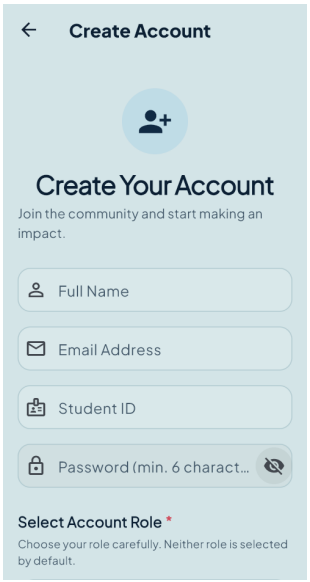
Code

```
AnimatedContainer(
  duration: const Duration(milliseconds: 200),
  decoration: BoxDecoration(
    color: isSelected ? AppColors.accentSoft : AppColors.surface,
    border: Border.all(color: isSelected ? AppColors.primary : AppColors.border, width: isSelected
? 2 : 1),
  ),
  child: /* role details */,
)

AnimatedScale(
  scale: _pressed ? 0.97 : 1,
  duration: const Duration(milliseconds: 120),
  curve: Curves.easeOut,
  child: widget.child,
)
```

Output

Choosing the Volunteer role animates its card to a highlighted, outlined state while the Head Volunteer card stays neutral.



Signup form



Role card animated to selected

Experiment 9(a): Fetch data from a REST API

Aim

To fetch data from a REST API using examples relevant to the Volunteer Connect project.

Description

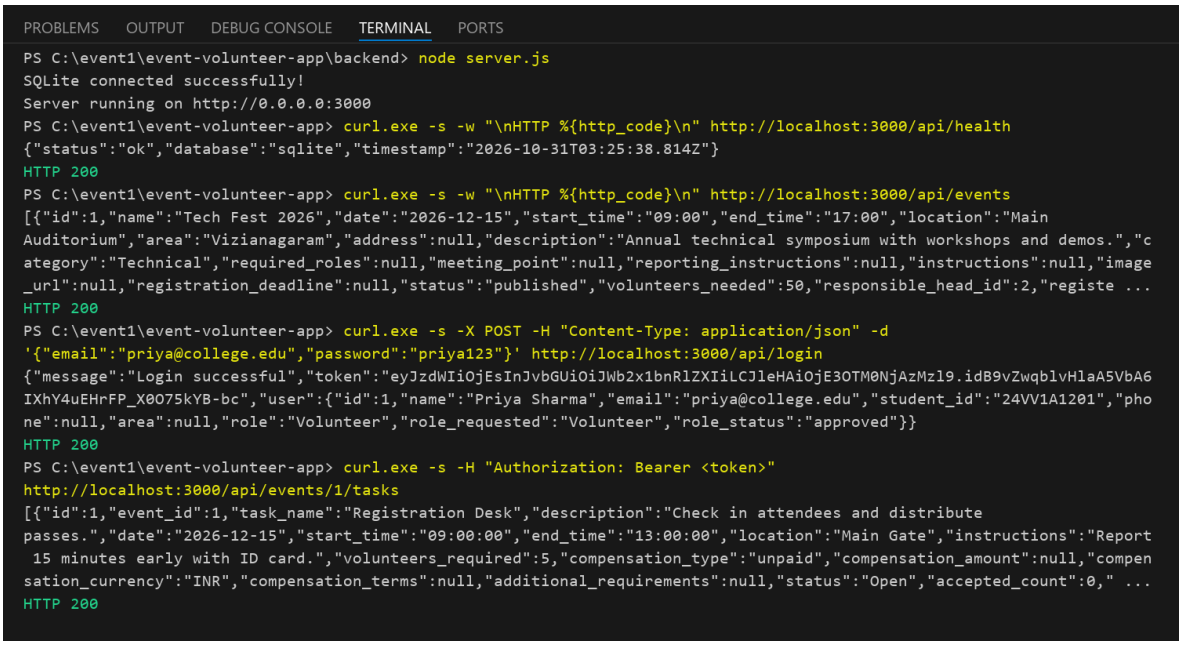
A REST API lets the application request or submit information through HTTP endpoints. Volunteer Connect talks to its own Node.js and Express server at http://localhost:3000 through the ApiService class, which uses the http package, adds a bearer token header after login, applies a 15-second timeout and converts network errors into readable messages. getEvents sends GET /api/events with optional area, category, date and availability filters, decodes the JSON list and caches it in the local SQLite database. The responses below were obtained from the running server. (The Android build uses an on-device implementation of the same routes, so the phone needs no server; the Node.js server is used here to demonstrate the REST API.)

Code

```
 1  static Future<List<dynamic>> getEvents({String? area, String? category, String? date, bool
availableOnly = false}) async {
 2    final query = <String, String>{
 3      if (area != null && area.trim().isNotEmpty) 'area': area.trim(),
 4      if (availableOnly) 'available': 'true',
 5    };
 6    final eventsUri = uri('/api/events').replace(queryParameters: query.isEmpty ? null : query);
 7    final response = await _send(() => http.get(eventsUri, headers: _headers));
 8    if (response.statusCode == 200) {
 9      final decoded = jsonDecode(response.body);
10      if (decoded is List) { unawaited(LocalDatabaseService.instance.cacheEvents(decoded)); return
decoded; }
11    }
12    throw ApiException('Failed to load events', statusCode: response.statusCode);
13  }
```

Output

The backend was started in the VS Code terminal and four calls were made, all returning HTTP 200: GET /api/health, GET /api/events, POST /api/login and GET /api/events/1/tasks with the bearer token. Long JSON bodies are cut with "..." in the capture.



REST API calls and the data received, shown in the VS Code terminal

Experiment 9(b): Display fetched data meaningfully in the UI

Aim

To display fetched data meaningfully in the UI using examples relevant to the Volunteer Connect project.

Description

Data fetched from an API should be presented as a readable interface rather than raw JSON. The Events page shows a CircularProgressIndicator while the request is running, a LoadErrorState with a Retry button if it fails, an empty-state message when there are no events, and otherwise maps every JSON object to an EventCard. Dates are converted by formatDate ("Dec 15, 2026"), start times by formatTime ("9:00 AM"), and available_slots becomes the "50 spots" or "Full" badge. Selecting View event opens the details screen with the tasks fetched for that event.

Code

```
if (isLoading) const Center(child: CircularProgressIndicator())
else if (loadError != null) LoadErrorState(message: loadError!, onRetry: loadEvents)
else if (events.isEmpty) const Text('No events available.')
else ...events.map((event) => EventCard(
  eventId: (event['id'] as num).toInt(),
  eventName: event['name'] ?? 'Unnamed Event',
  date: formatDate(event['date']),
  location: event['location'] ?? '',
  volunteersNeeded: (event['available_slots'] as num?)?.toInt(),
  startTime: event['start_time']?.toString() ?? '',
))
```

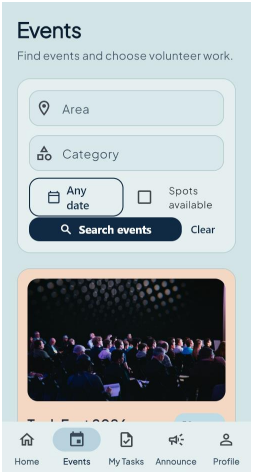
Output

The JSON returned by GET /api/events is first read as a table in the terminal (id, name, date, location, category and free spots), and the same three events then appear in the app as cards and as a detailed page with organiser contact and registration controls.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\event1\event-volunteer-app> Invoke-RestMethod http://localhost:3000/api/events | ForEach-Object { $_ } |
Select-Object id,name,date,location,category,available_slots | Format-Table

id name                date       location             category  available_slots
-- ----                ----       --------             --------  ---------------
 1 Tech Fest 2026      2026-12-15 Main Auditorium      Technical             50
 2 Blood Donation Camp 2026-12-20 College Health Centre Health                30
 3 Cultural Night      2027-01-10 Open Air Theatre     Cultural              40
```

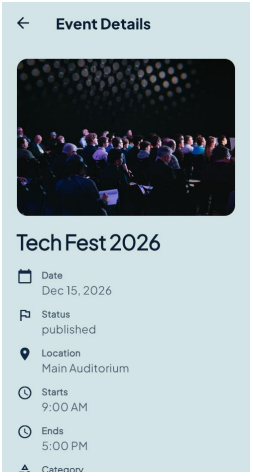
Fetched event data displayed in the VS Code terminal



Events list



Second event



Event details

Experiment 10(a): Write unit tests for UI components

Aim

To write unit tests for UI components using examples relevant to the Volunteer Connect project.

Description

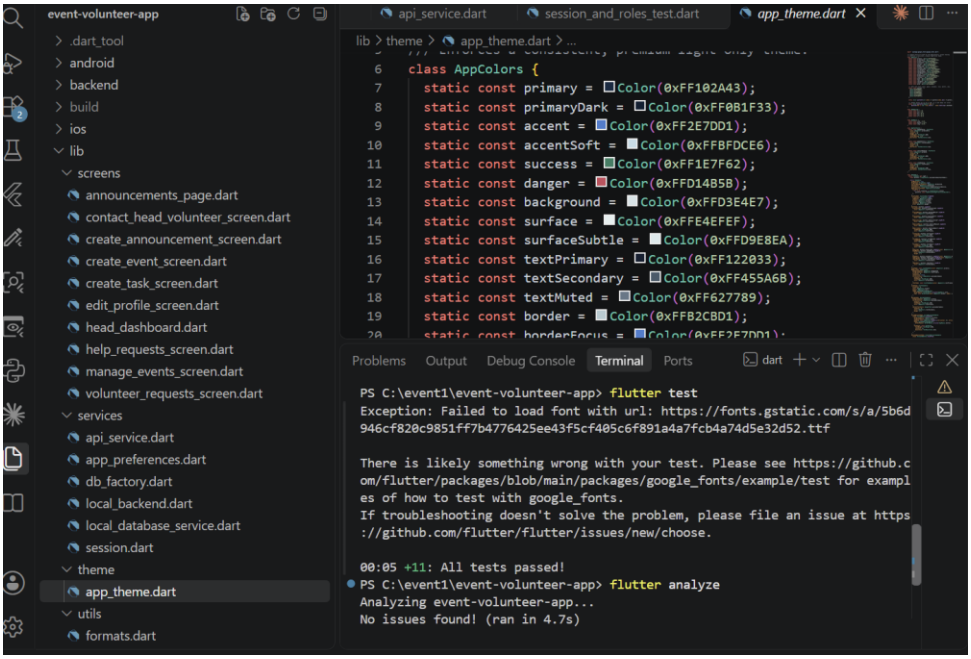
Unit and widget tests verify that important behaviour stays correct after code changes. The test folder of Volunteer Connect contains six test files: app_preferences_test.dart (light theme values and the stored API base URL), local_database_test.dart (SQLite cache operations), session_and_roles_test.dart (permissions derived from the logged-in role), navigation_test.dart (a widget test that taps the Home quick actions and checks the selected tab), local_backend_test.dart (drives the on-device backend through register, login, event, task, enrollment, approval, announcement and help-request calls) and widget_test.dart. Running flutter test executed 11 tests and all passed. During the widget tests, google_fonts printed an error because it could not download the font without network access to fonts.gstatic.com; the tests still passed.

Code

```
testWidgets('Home quick actions navigate through the persistent tabs', (tester) async {
  Session.clear();
  await tester.pumpWidget(MaterialApp(theme: AppTheme.light,
    home: const VolunteerHomeScreen(user: {'name': 'Test Volunteer'})));
  expect(find.text('Hello, Test Volunteer \u{1F44B}'), findsOneWidget);
  await tester.tap(find.text('View Events').first);
  await tester.pump();
  expect(find.text('Find events and choose volunteer work.'), findsOneWidget);
});
```

Output

Result of flutter test and flutter analyze in the VS Code terminal: all 11 tests passed and the analyzer found no issues. The font-download error above the result is the google_fonts message described above.



flutter test and flutter analyze output in the VS Code terminal

Experiment 10(b): Use Flutter debugging tools to identify and fix issues

Aim

To use Flutter debugging tools to identify and fix issues using examples relevant to the Volunteer Connect project.

Description

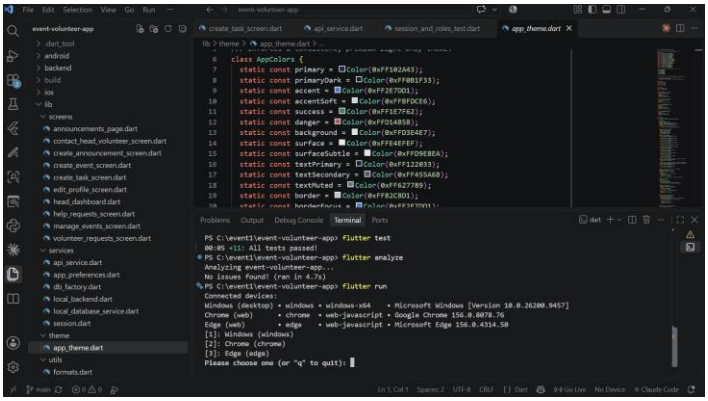
flutter analyze reports static-analysis problems, while the debug build shows runtime layout problems directly on the screen. flutter analyze on the project reported no issues. While the app was run in debug mode on the Windows desktop target, a runtime layout problem was found that the analyzer cannot detect: on the Contact Head Volunteer and New Announcement screens, the "Event" drop-down row shows the yellow-black strip "OVERFLOWED BY 82 PIXELS" at the window width used for the captures. This issue was observed and recorded; it has not been fixed in the submitted code, so it is listed as a known issue rather than a resolved one.

Code

```
flutter analyze
flutter test
flutter run -d windows
# DevTools > Flutter Inspector > select the Event drop-down to inspect its width constraints
# Candidate fix to try: wrap the drop-down in SizedBox(width: double.infinity) and set isExpanded: true
```

Output

Left: the VS Code terminal shows flutter test (all passed), flutter analyze ("No issues found!") and flutter run offering the Windows, Chrome and Edge targets. Right: the runtime overflow strip on the New Announcement screen; the Contact Head Volunteer screen (Screen 10, mobile layout) shows the same strip.



VS Code terminal: test, analyze and run



New Announcement: overflow strip

Case Study: Volunteer Connect (Event Volunteer Management System)

Introduction

Volunteer Connect is a Flutter application, with a Node.js and Express backend and a SQLite or MySQL database, for organising volunteers at college events. Students register as Volunteers and browse published events, enroll in volunteer tasks and follow their status. Head Volunteers create events and tasks, review volunteer applications, publish announcements and answer help requests. The application was run on the Windows desktop target and communicates with the backend over HTTP. The Android build runs standalone: an on-device SQLite database answers the same routes, so the phone needs no server.

Problem Statement

Volunteer coordination for college events is often handled through chat groups and spreadsheets, which makes it hard to see how many volunteers are still needed, who has been accepted and which updates have been sent. Volunteer Connect provides one structured place for events, tasks, applications and announcements, with different capabilities for volunteers and head volunteers.

Objectives

- Provide registration and login with role selection (Volunteer or Head Volunteer).
- Let volunteers browse events and filter them by area, category, date and spots available.
- Allow volunteers to register for an event and enroll in tasks without duplicate enrollments.
- Show enrollment status (pending, approved, declined) and event announcements.
- Let Head Volunteers create and manage events and tasks, review applicants, post announcements and respond to help requests.
- Protect private routes with signed tokens and role checks, and store passwords as bcrypt hashes.
- Cache event data locally with SQLite so the app can show data when the server is unreachable.
- Demonstrate a consistent, light-themed UI with animation on Windows desktop.

Technology Stack

Technology	Use in Volunteer Connect
Flutter / Dart	Cross-platform UI framework and application language.
Node.js + Express	REST API server (backend/server.js).
SQLite / MySQL	Server database (SQLite by default; MySQL optional).
sqflite_common_ffi	Local SQLite cache on the desktop client.
http, shared_preferences	API calls and local preferences (API base URL).
google_fonts	Plus Jakarta Sans typography.
bcrypt, signed tokens	Password hashing and authenticated sessions.
VS Code, Flutter DevTools	Development and debugging.

System Architecture

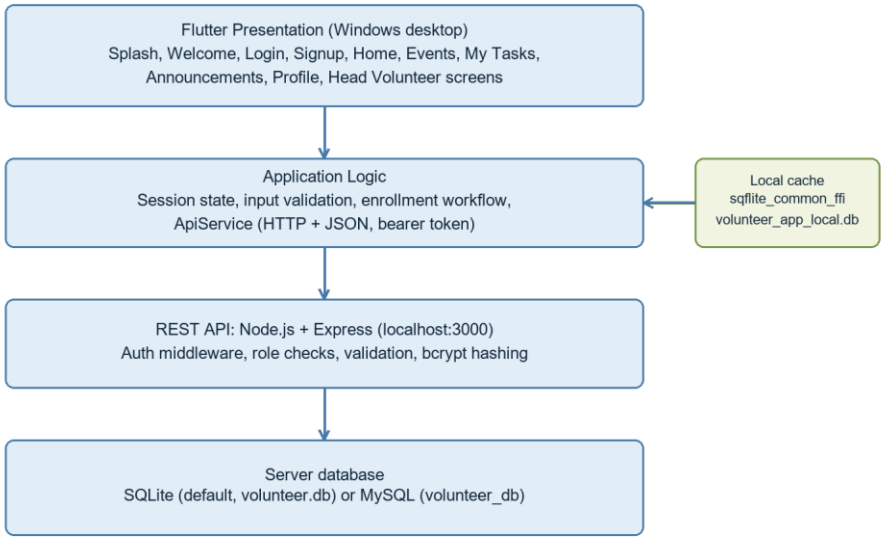


Figure 1: Volunteer Connect high-level architecture

Database Design

The server database holds six core tables: users, events, tasks, enrollments, announcements and help_requests. The SQLite store also creates event_registrations, notifications and event_history tables for event sign-ups, in-app updates and an audit trail. The description below is taken from the schema in backend/server.js; verify it against the final file before submission.

Table	Purpose and key columns
users	Account and role: name, email (unique), student_id, hashed password, role, role_requested, role_status.
events	Event name, date, start/end time, location, area, category, description, volunteers_needed, status (draft/published), responsible head volunteer.
tasks	Volunteer tasks of an event: task_name, date, times, location, instructions, volunteers_required, compensation fields.
enrollments	A volunteer's application to a task: user_id, task_id, status (pending/approved/rejected); unique per user and task.
announcements	Event updates: event_id, title, message, created_at.
help_requests	Questions to the head volunteer: user_id, event_id, message, status (open/resolved).

Database Relationships

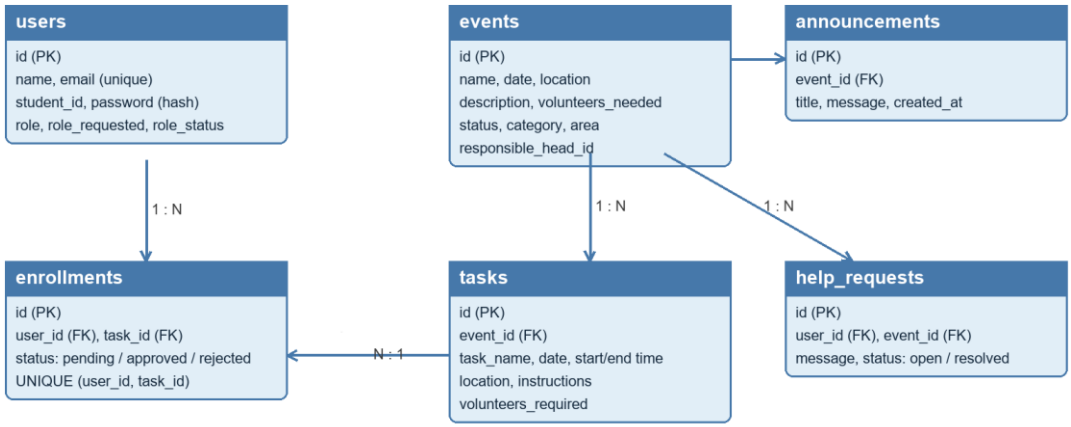


Figure 2: Logical Volunteer Connect database relationships

Key Rules Enforced by the Backend

- Duplicate enrollment in the same task returns HTTP 409 (UNIQUE user_id, task_id).
- Self-registration creates Volunteer accounts only; Head Volunteer access is granted by a database administrator.
- Private routes require a bearer token (requireAuth); event, task, enrollment, announcement and help-request management additionally require event-staff rights.
- Deleting an event cascades to its tasks and announcements (foreign keys with ON DELETE CASCADE).
- Event publishing is checked against a publishing deadline; times must satisfy start < end.

REST Endpoints Used by the Application

Method	Endpoint	Purpose
POST	/api/register, /api/login	Create account, obtain token
GET	/api/events, /api/events/:id	List / read events (filters supported)
POST/PUT	/api/events, /api/events/:id/publish	Create, edit, publish events (staff)
POST	/api/events/:id/register	Register for an event
GET/POST	/api/events/:id/tasks, /api/tasks	List tasks, create task (staff)
POST	/api/enroll	Enroll in a task
GET	/api/users/:id/tasks	My Tasks
GET/PUT	/api/enrollments, /api/enrollments/:id/status	Review applicants (staff)
GET/POST	/api/announcements	Read / publish announcements
GET/POST	/api/help-requests	Submit / review help requests

User Workflow



Figure 3: Volunteer workflow

Head Volunteer Workflow

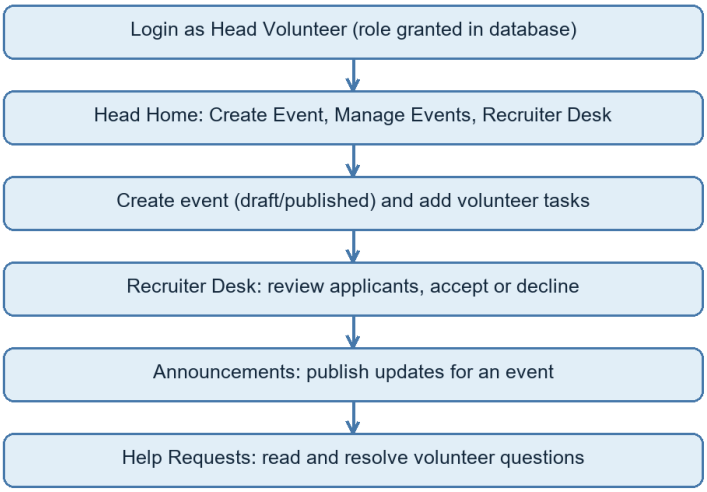


Figure 4: Head Volunteer workflow

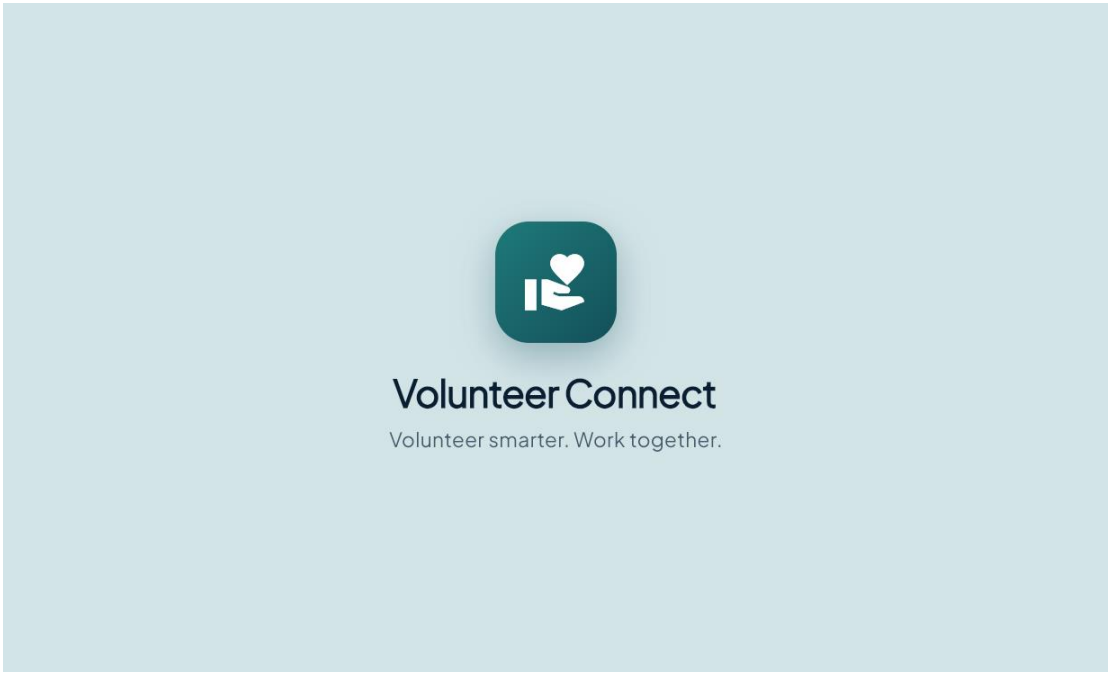
Screen-by-Screen Output Gallery

The following screens were captured from the running application (Windows desktop build with demonstration data). Each screen is shown twice: in a mobile layout (a narrow, phone-shaped window) and in a desktop layout (a wide window). Volunteer screens come first, followed by the Head Volunteer screens.

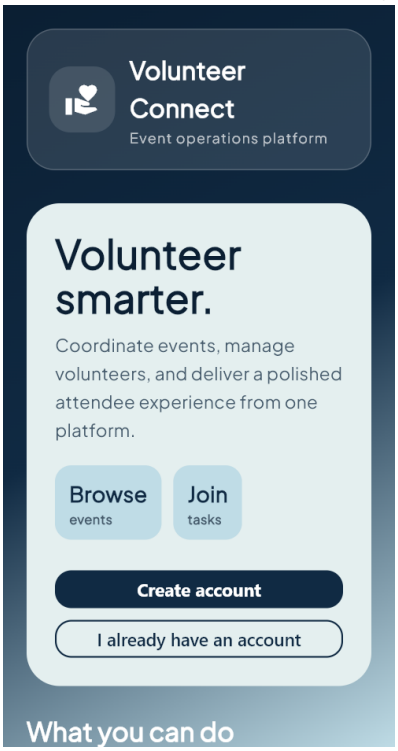
Screen 1: Splash screen (mobile layout).



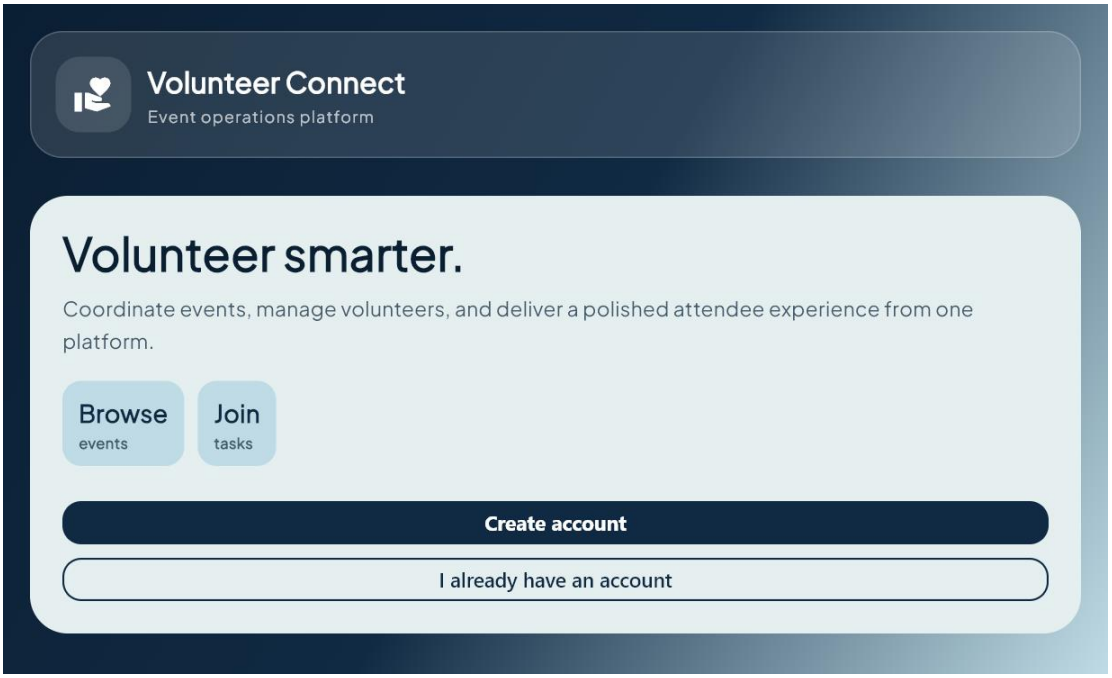
Screen 1: Splash screen (desktop layout).



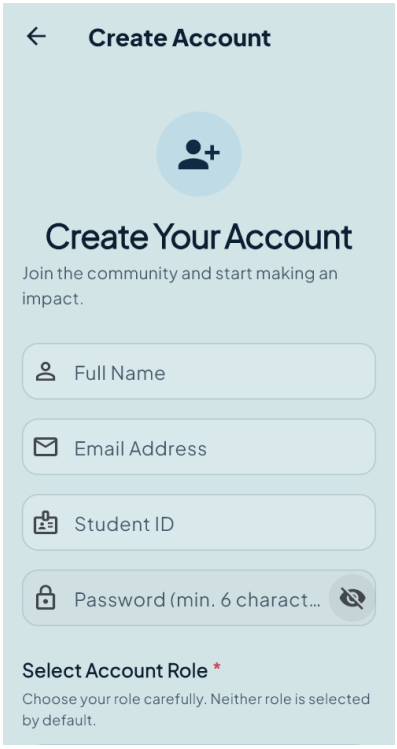
Screen 2: Welcome screen (mobile layout).



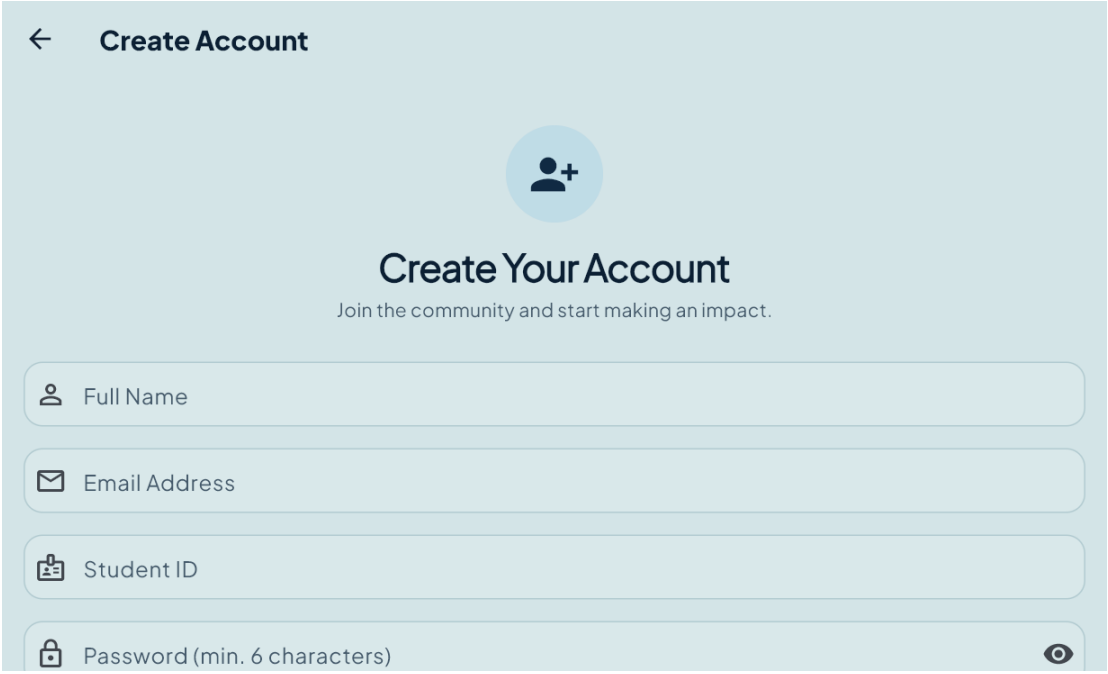
Screen 2: Welcome screen (desktop layout).



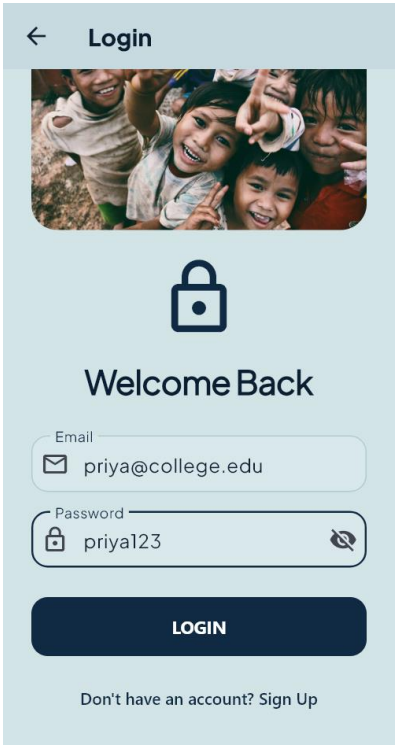
Screen 3: Create Account screen (mobile layout).



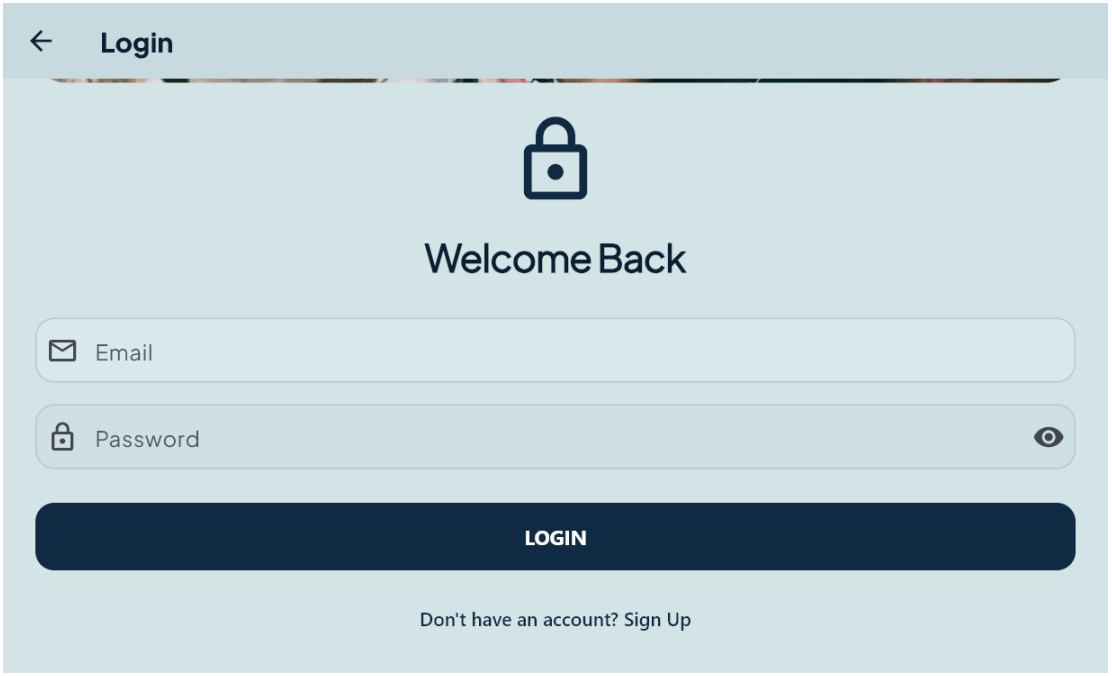
Screen 3: Create Account screen (desktop layout).



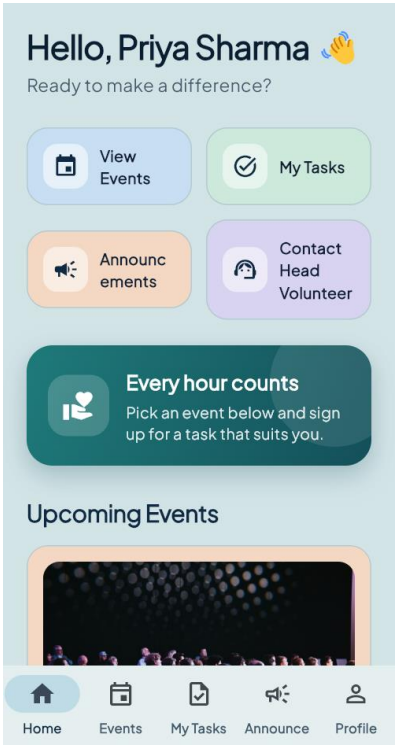
Screen 4: Login screen (mobile layout).



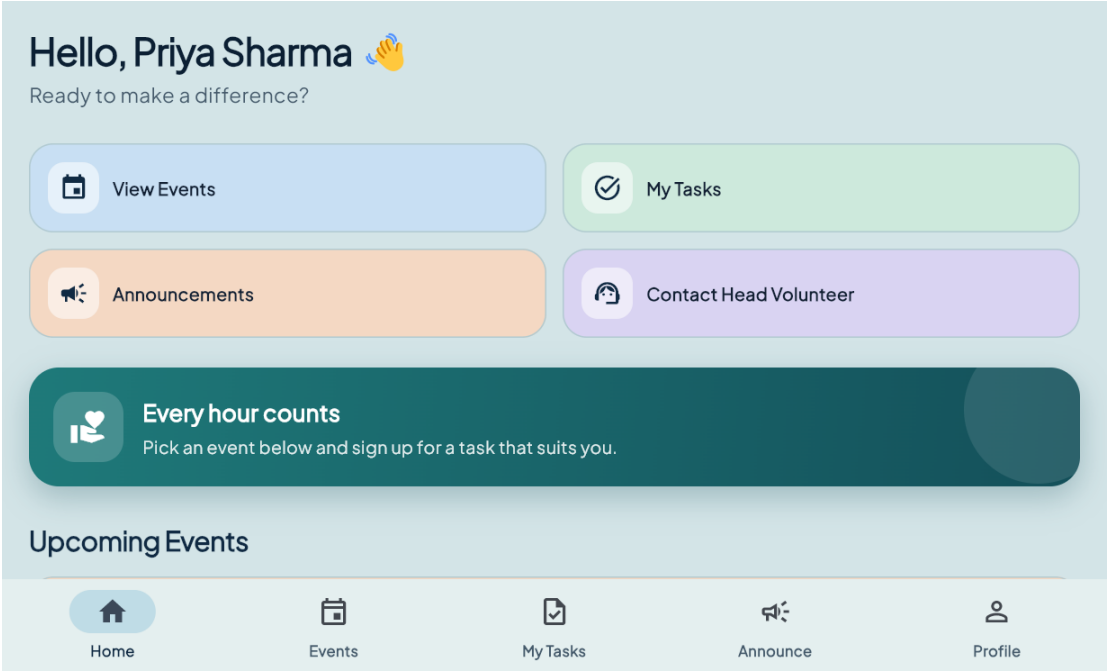
Screen 4: Login screen (desktop layout).



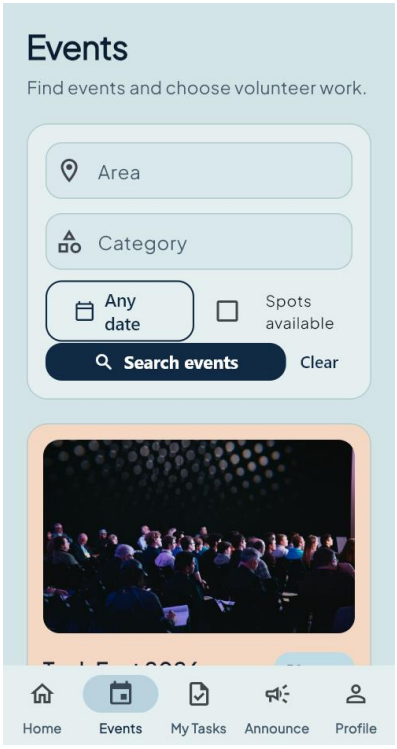
Screen 5: Volunteer Home screen (mobile layout).



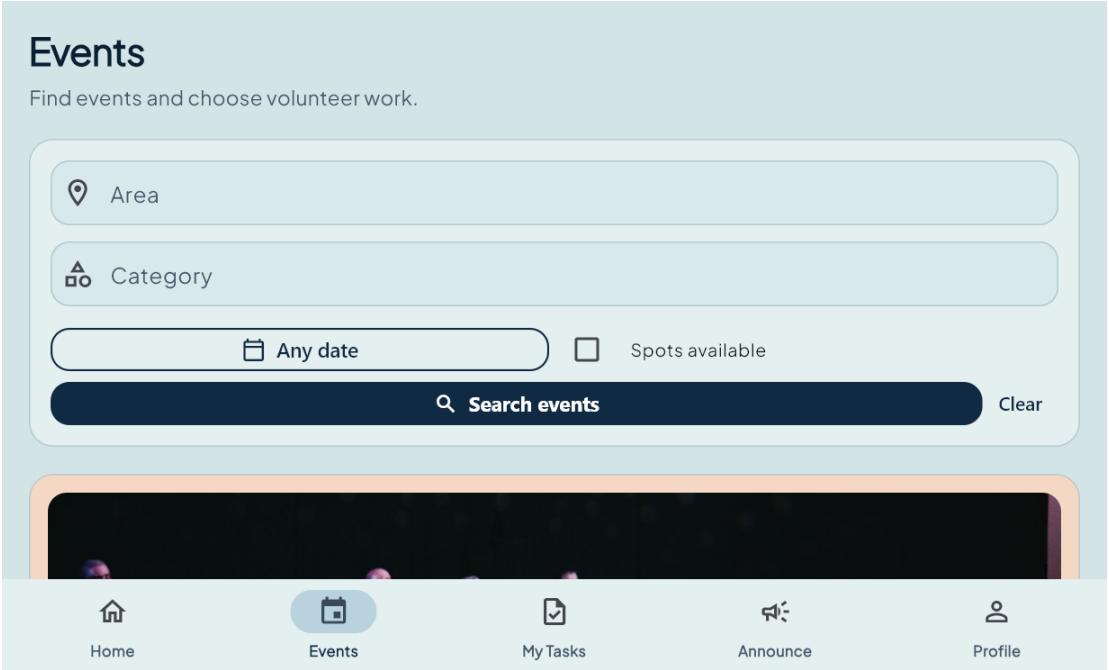
Screen 5: Volunteer Home screen (desktop layout).



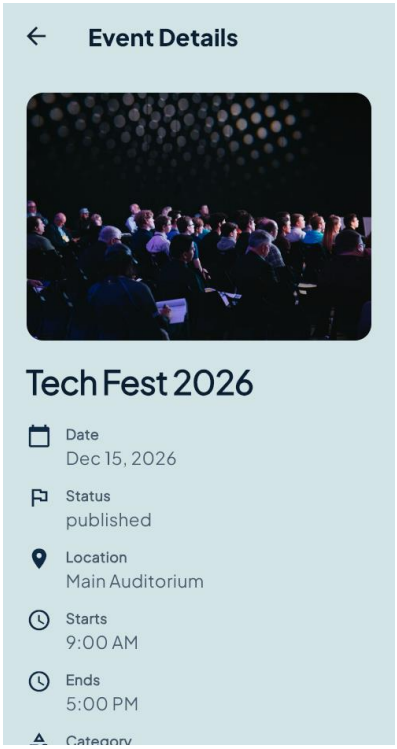
Screen 6: Events screen (mobile layout).



Screen 6: Events screen (desktop layout).



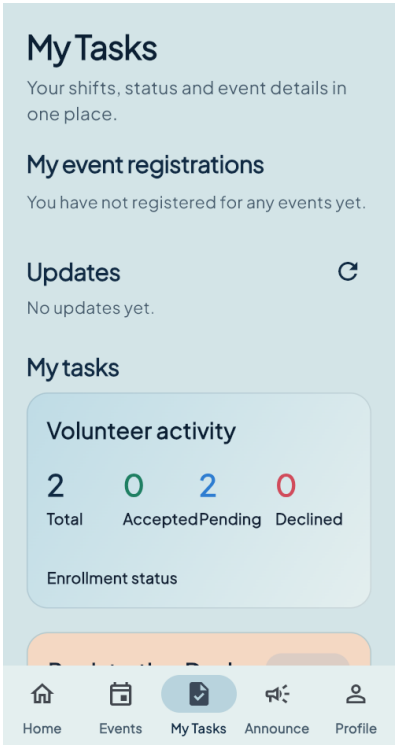
Screen 7: Event Details screen (mobile layout).



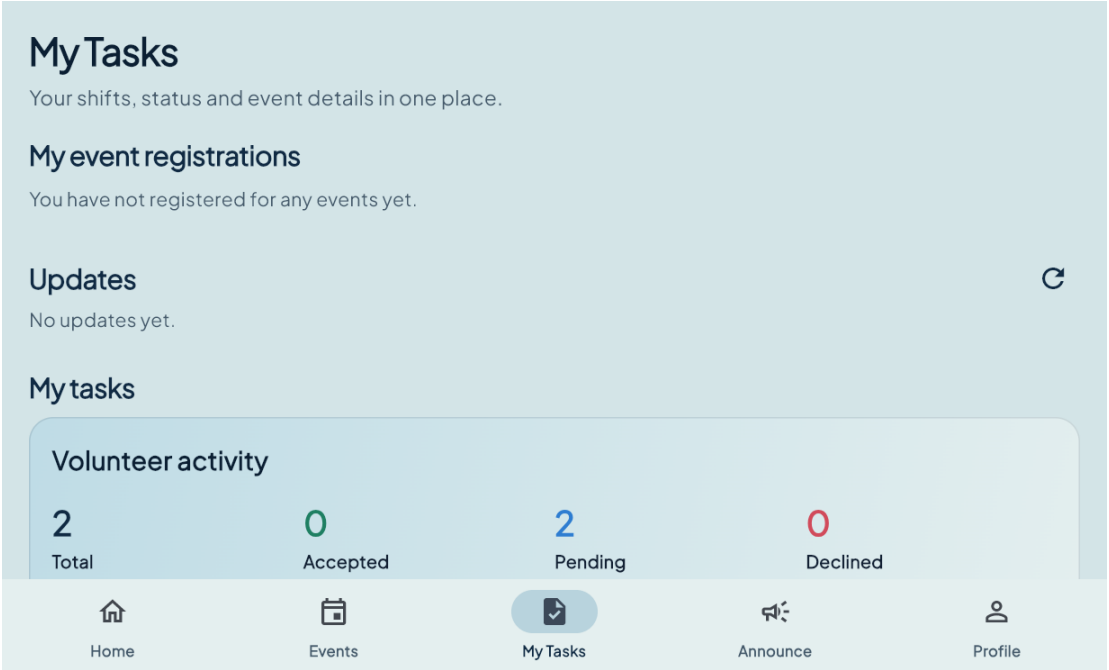
Screen 7: Event Details screen (desktop layout).



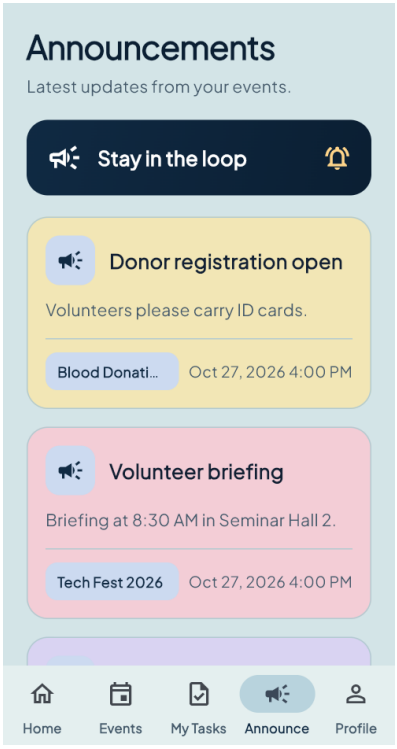
Screen 8: My Tasks screen (mobile layout).



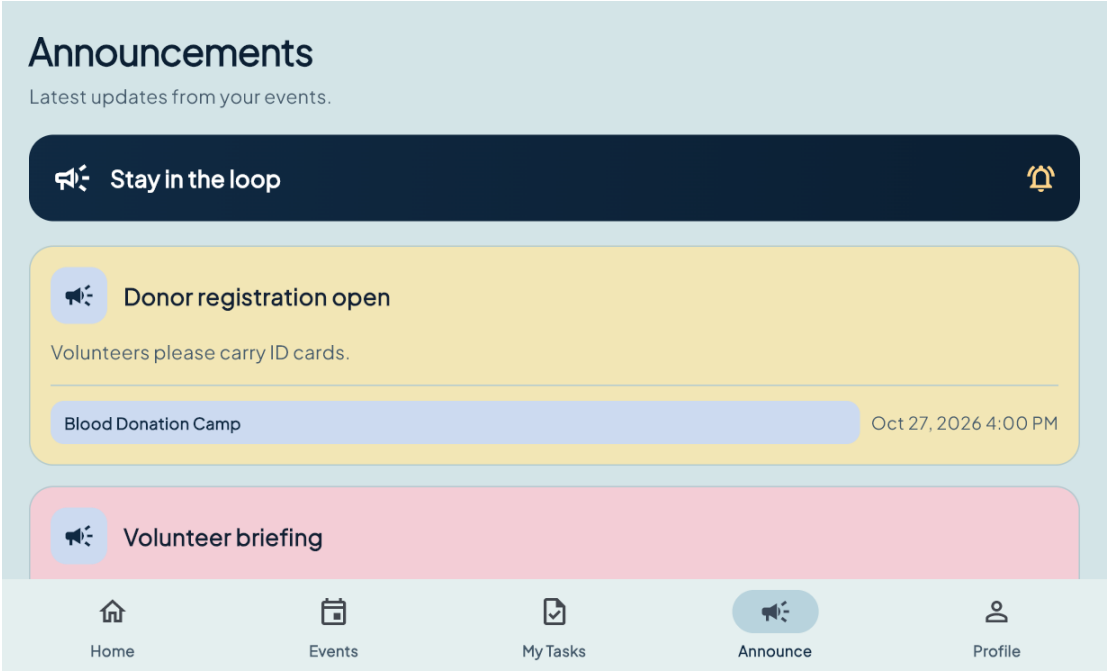
Screen 8: My Tasks screen (desktop layout).



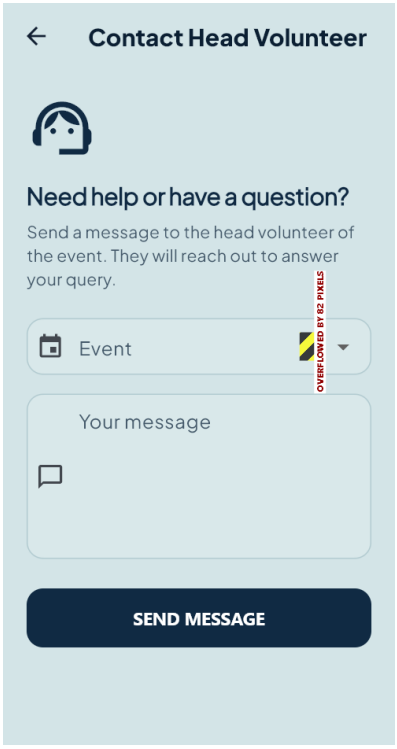
Screen 9: Announcements screen (mobile layout).



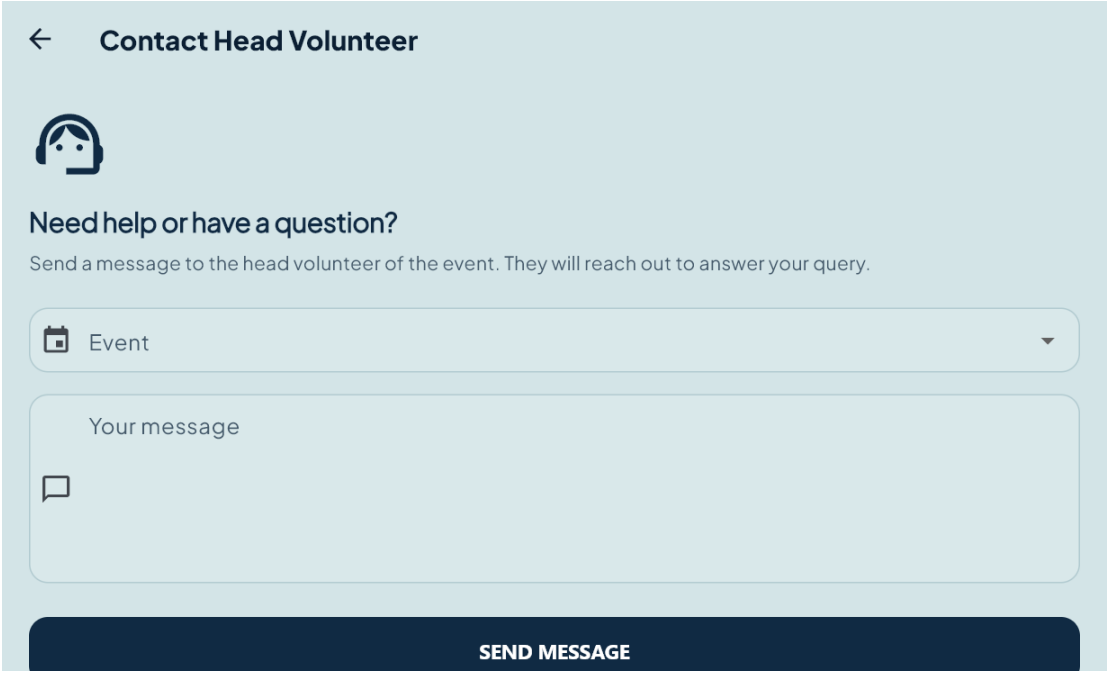
Screen 9: Announcements screen (desktop layout).



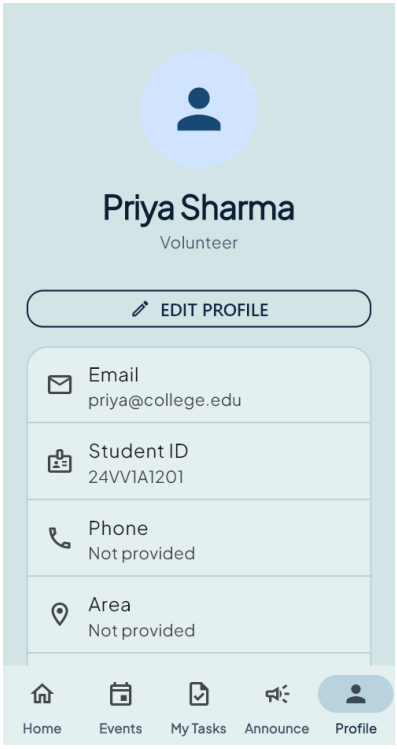
Screen 10: Contact Head Volunteer screen (mobile layout).



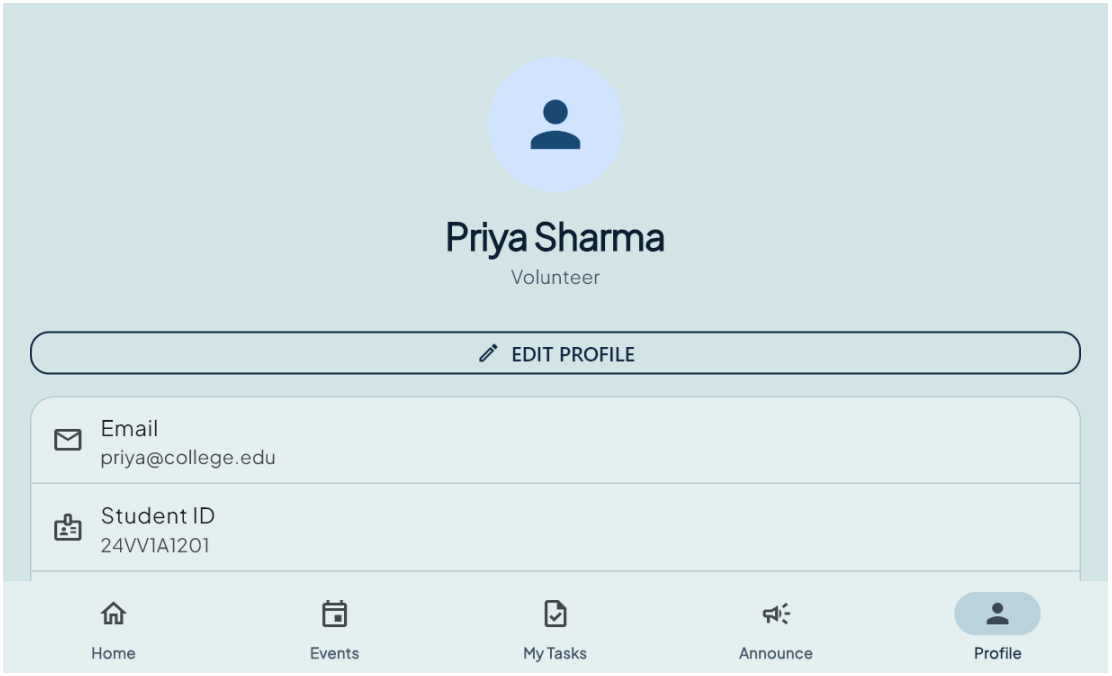
Screen 10: Contact Head Volunteer screen (desktop layout).



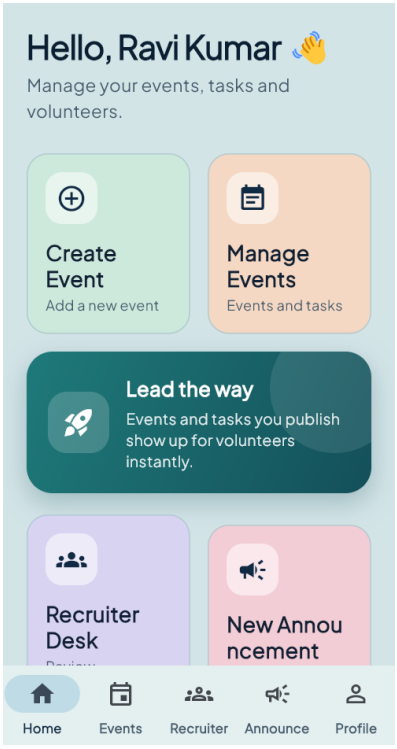
Screen 11: Volunteer Profile screen (mobile layout).



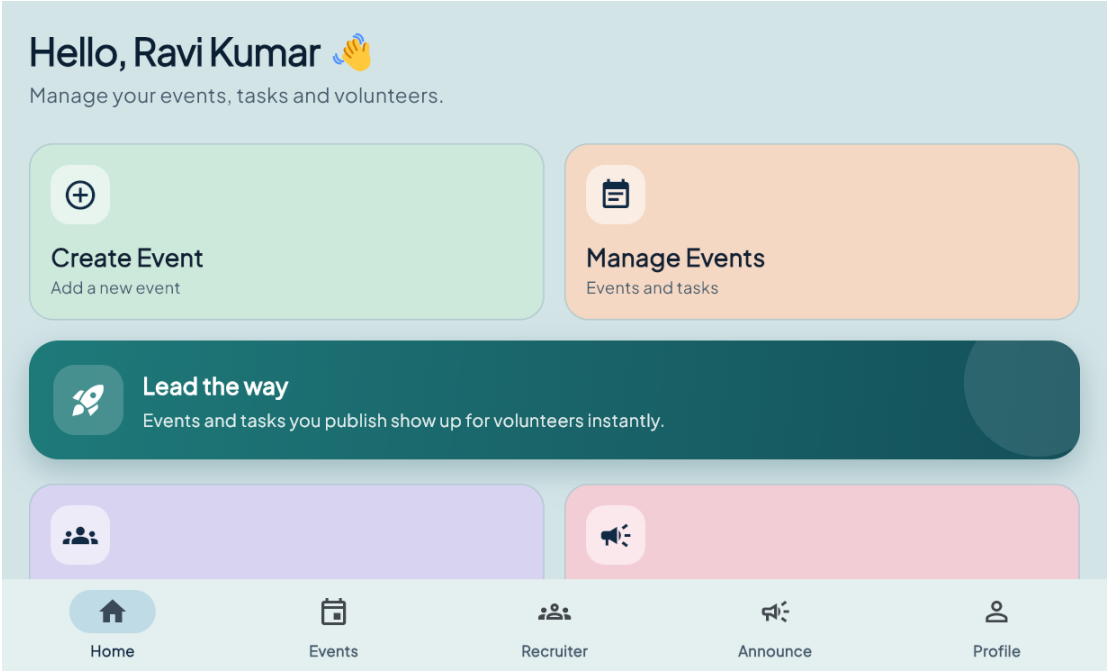
Screen 11: Volunteer Profile screen (desktop layout).



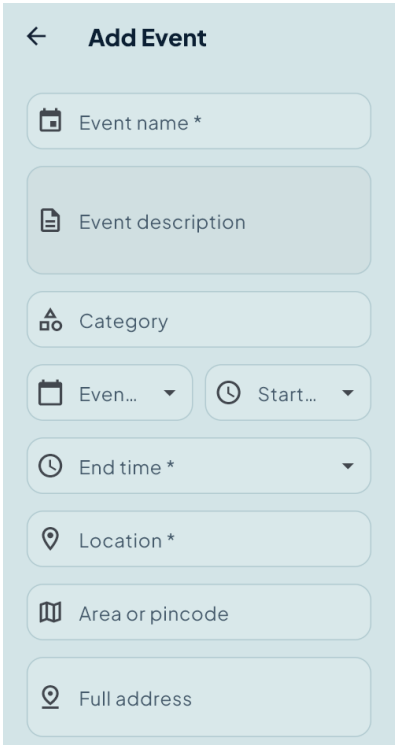
Screen 12: Head Volunteer Home screen (mobile layout).



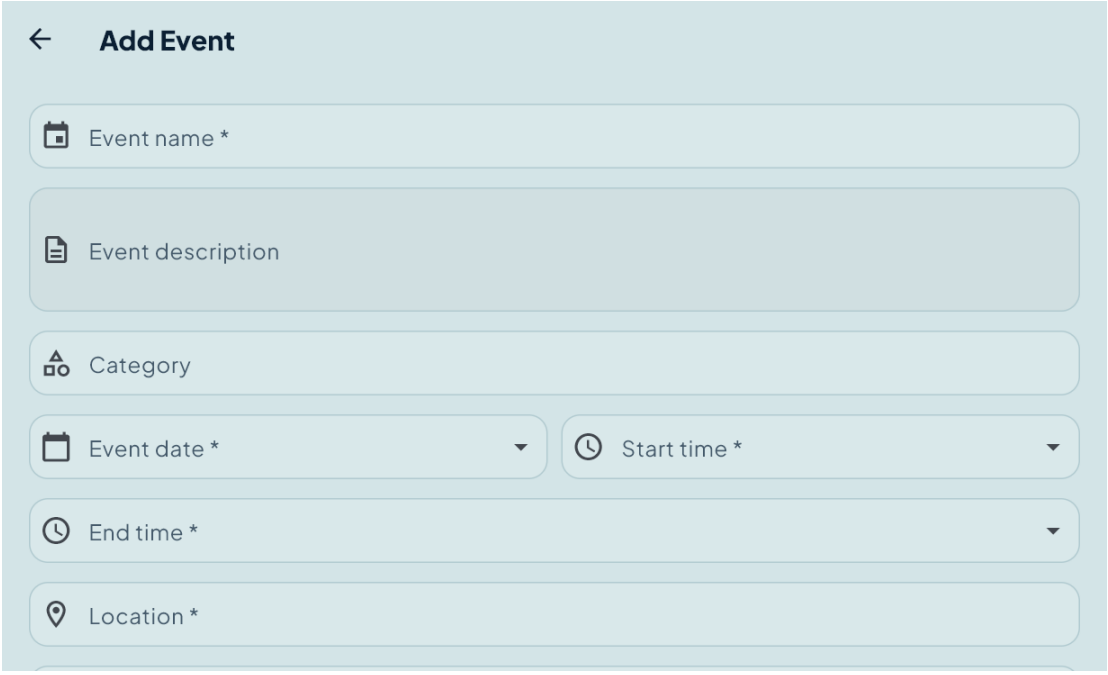
Screen 12: Head Volunteer Home screen (desktop layout).



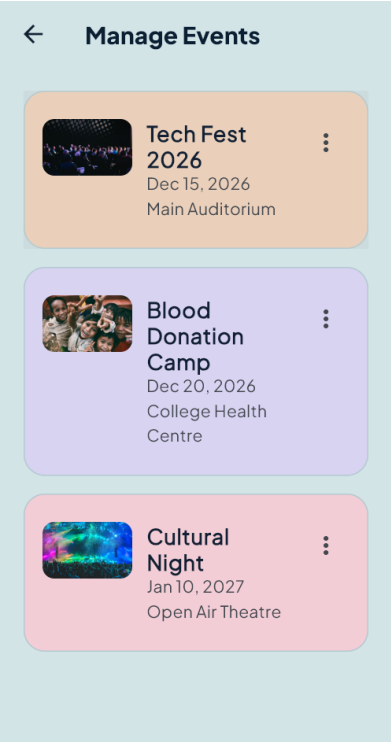
Screen 13: Add Event screen (mobile layout).



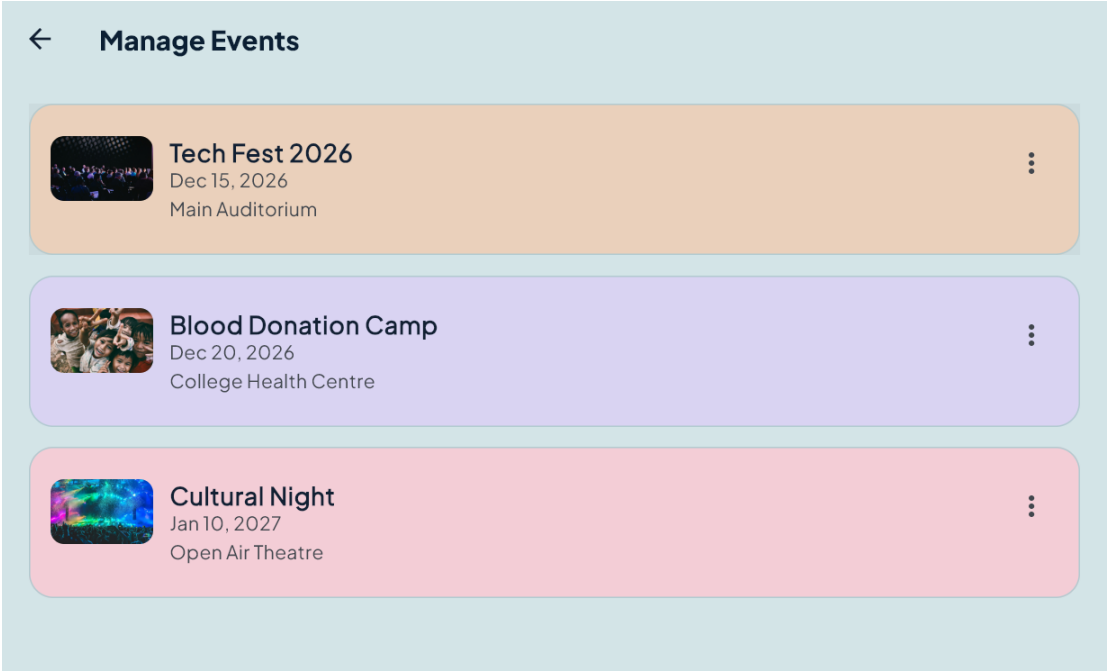
Screen 13: Add Event screen (desktop layout).



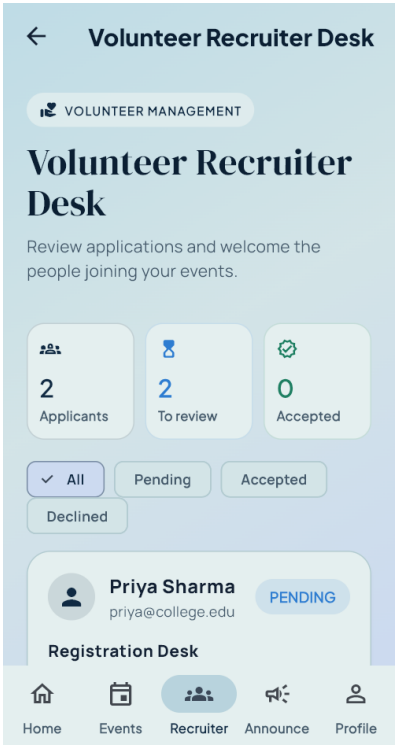
Screen 14: Manage Events screen (mobile layout).



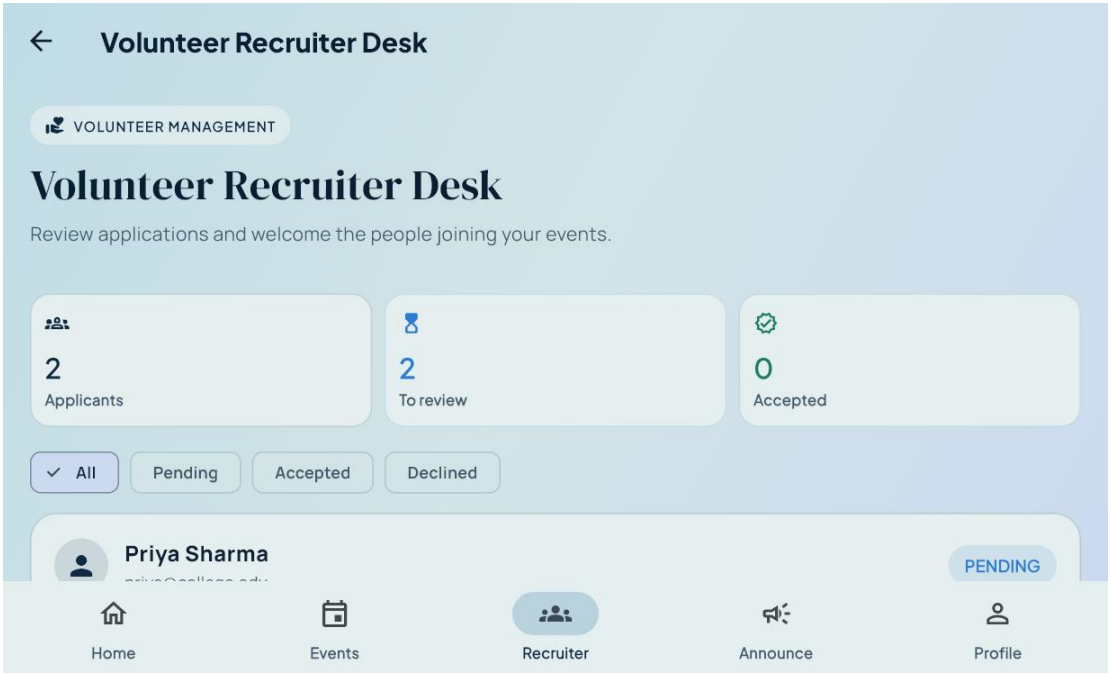
Screen 14: Manage Events screen (desktop layout).



Screen 15: Volunteer Recruiter Desk screen (mobile layout).



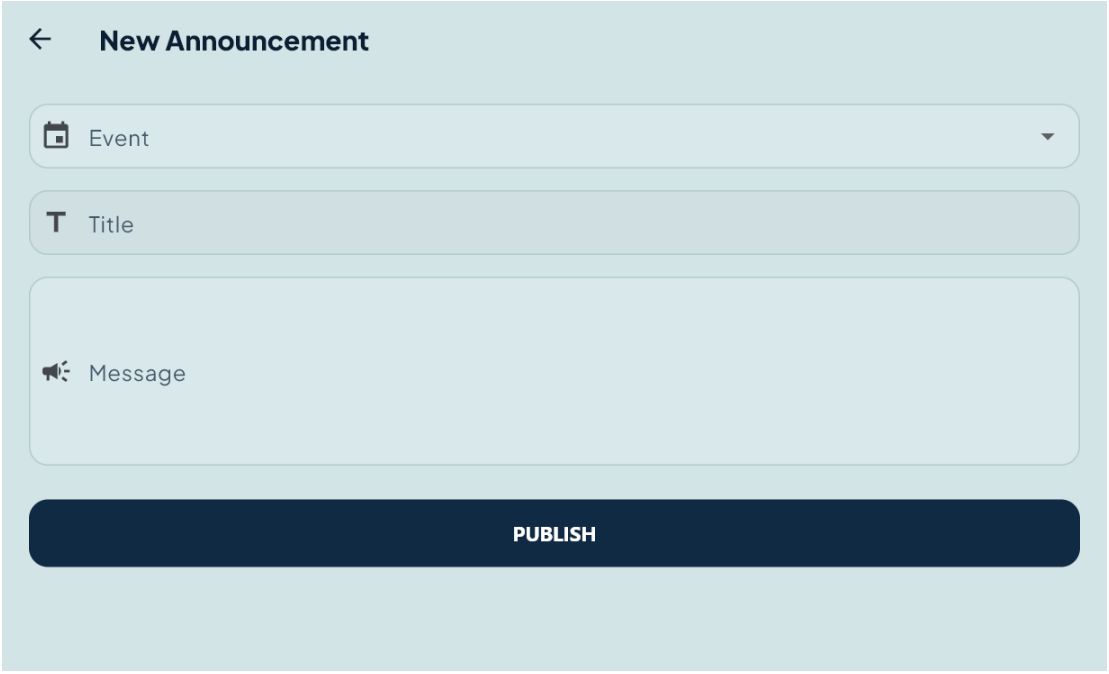
Screen 15: Volunteer Recruiter Desk screen (desktop layout).



Screen 16: New Announcement screen (mobile layout).



Screen 16: New Announcement screen (desktop layout).



Window-Size Comparison and Security Considerations

Only the Windows desktop build was captured for this record. The same screens were viewed in a narrow, phone-like window and in a wide window to compare how the layout responds.

Aspect	Narrow window	Wide window
Screen shape	Portrait, similar to a phone; content scrolls vertically.	Landscape; cards and fields stretch to the full width.
Navigation	Bottom NavigationBar with five tabs.	Same bar, with the tabs spread across the width.
Layout	Single column; chips wrap onto new lines.	Single column; no breakpoint re-arrangement (see Exp. 3(b)).
Storage	Server database plus local SQLite cache.	Same.

Security and Privacy Considerations

- Passwords are stored as bcrypt hashes; legacy plain-text values are converted at backend start-up. The client does not keep passwords in shared_preferences.
- Login returns a signed, short-lived bearer token; private routes check it and the account role.
- Head Volunteer privileges cannot be self-assigned from the client and are granted in the database.
- Input is validated in the app and again on the server. A custom AUTH_TOKEN_SECRET of at least 32 characters should be set in backend/.env for any real deployment.

Testing Plan and Results

The tests below were carried out on the running application with the demonstration data. "Observed" means the behaviour was seen in the captured run; "Not executed" means it was not exercised for this record.

ID	Test	Expected result	Result / evidence
T01	Submit empty signup form	Validation message shown	Observed: Exp. 7(b)
T02	Login with valid volunteer account	Volunteer Home opens	Observed: Screen 5
T03	Submit empty login form	Validation message shown	Observed: Exp. 7(b)
T04	Browse events	Published events listed with spots	Observed: Screen 6
T05	Open event details	Date, time, organiser and tasks shown	Observed: Screen 7
T06	Enroll in a task	Task appears in My Tasks as Pending	Observed: Screen 8 (enrollment made through the API)
T07	Login as Head Volunteer	Head Home with Recruiter tab	Observed: Screen 12
T08	Review applicants	Pending applicant listed in Recruiter Desk	Observed: Screen 15
T09	View announcements	Event announcements listed	Observed: Screen 9
T10	Accept / decline an applicant	Status changes and volunteer sees update	Not executed
T11	Create an event from the Add Event form	Event saved and listed	Not executed (events were seeded through the API)
T12	Submit a help request	Request appears in Help Requests	Not executed
T13	Run flutter analyze	No analyzer issues	Observed: "No issues found!"
T14	Run flutter test	All tests pass	Observed: 11 tests passed
T15	Check layout in debug mode	No overflow errors	Failed in mobile layout: overflow of 82 px on drop-down (Exp. 10(b))

Limitations

- This record covers the Windows desktop build only; Android and web builds were not run or captured.
- In remote mode (API_BASE_URL set) the backend must be running at that URL; the standalone build keeps its own data on each device and does not synchronise between devices.
- Provider is not used, and navigation uses MaterialPageRoute instead of named routes; Experiments 3(b) and 4(b) show proposed code for the missing parts.
- The layout does not rearrange into multiple columns on wide windows.
- The Event drop-down on the Contact Head Volunteer and New Announcement screens overflows at the captured window width (Experiment 10(b)).
- Event artwork is loaded from the internet; offline, a fallback icon is shown instead.
- The application uses a single light theme; there is no dark mode.
- Accepting applicants, creating events through the form and submitting help requests were not exercised in this capture.
- Flutter widget tests try to download Google Fonts and print an error when offline, although they pass.

Conclusion

Volunteer Connect demonstrates the main ideas of the Flutter lab in one working application: widgets and layouts, themes and custom widgets, navigation, state with setState and a Session class, forms and validation, animations, REST API integration with a Node.js backend, local SQLite caching, and unit and widget tests. Volunteers can register, browse and filter events, enroll in tasks and follow their status, while Head Volunteers can manage events and volunteers from the same application. The captured run, the passing analyzer and test results, and the recorded overflow issue give an honest picture of the current state of the project.

Future Scope

- Fix the drop-down overflow and add responsive multi-column layouts with LayoutBuilder.
- Add Android and Chrome targets and capture those builds.
- Introduce Provider (or another state-management package) for shared state.
- Add push notifications for enrollment decisions and announcements.
- Add attendance tracking, volunteer-hour reports and dark mode.

References

1. Flutter Documentation, https://docs.flutter.dev/.
2. Dart Documentation, https://dart.dev/guides.
3. SQLite Documentation, https://www.sqlite.org/docs.html.
4. sqflite package, https://pub.dev/packages/sqflite.
5. sqflite_common_ffi package, https://pub.dev/packages/sqflite_common_ffi.
6. http package, https://pub.dev/packages/http.
7. google_fonts package, https://pub.dev/packages/google_fonts.
8. shared_preferences package, https://pub.dev/packages/shared_preferences.
9. crypto package, https://pub.dev/packages/crypto.
10. Node.js Documentation, https://nodejs.org/en/docs.
11. Express Routing Guide, https://expressjs.com/en/guide/routing.html.
12. Material Design 3, https://m3.material.io/.
13. Android Developers, Run apps on a hardware device, https://developer.android.com/studio/run/device.